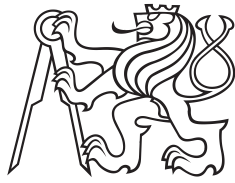


Master Thesis



Czech
Technical
University
in Prague

F3

Faculty of Electrical Engineering

Numerical Optimal Control for Minimum Lap Time

Bc. Richard Ciglanský

Supervisor: doc. Ing. Zdeněk Hurák, Ph.D.
May 2026

DECLARATION

I, the undersigned

Student's surname, given name(s): Ciglanský Richard
Personal number: 498930
Programme name: Cybernetics and Robotics

hereby declare that the Master's Thesis titled

Numerical Optimal Control for Minimum Lap Time

is my own independent work and that I have declared all sources of information used in accordance with the Methodological Guideline for adhering to ethical principles when elaborating an academic final thesis and the Framework Rules for the use of artificial intelligence at CTU for study and pedagogical purposes in Bachelor and continuing Masters studies.

I declare that I have used artificial intelligence tools during the preparation and writing of the final thesis.
I've verified the generated content. I confirm that I am aware that I am fully responsible for the content of the final thesis.

In Prague on 20.05.2026

Bc. Richard Ciglanský

.....
student's signature

Abstract

The thesis researches and implements ways of solving the minimum-time maneuvering problem (lap-time simulation) for road vehicles. It divides the problem into smaller subproblems for which mature software methods already exist. The subproblems are: race-car modeling, transcription of the optimal control problem into an NLP, solving the NLP, solving sets of linear equations, and sensitivity analysis. Each subproblem is examined in the context of vehicle dynamics, and a Julia-based implementation built on JuMP and Ipopt is presented. The framework supports several collocation schemes (Gauss-Legendre, Radau, and Lobatto) together with adaptive mesh refinement. Various vehicle models of increasing complexity, transcription strategies, and linear solver backends (MUMPS, HSL MA57/MA97) are benchmarked on representative circuits, and the parametric sensitivity of the optimal lap time is analyzed.

Keywords: NLP, optimization, automatic differentiation, twin-track model, Pacejka, Julia, scalability, Ipopt, MadNLP, MUMPS, MA97, CuDSS, sensitivity analysis

Supervisor: doc. Ing. Zdeněk Hurák, Ph.D.

Abstrakt

Táto práca skúma a implementuje metódy riešenia problému minimálneho času manévrovania (simulácia času na kolo) pre cestné vozidlá. Problém je rozdelený na menšie podproblémy, pre ktoré už existujú vyspelé softvérové nástroje. Týmito podproblémami sú: modelovanie pretekárskeho vozidla, transkripcia úlohy optimálneho riadenia na úlohu nelineárneho programovania (NLP), riešenie NLP, riešenie sústav lineárnych rovníc a citlivosť analýza. Každý podproblém je skúmaný v

kontexte dynamiky vozidla a je predstavená implementácia v jazyku Julia postavená na knižnici JuMP. Implementácia podporuje viacero kolokačných schém (Gauss-Legendre, Radau a Lobatto) spolu s adaptívnym zjemňovaním siete. Rôzne modely vozidiel rastúcej zložitosti, stratégie transkripcie a backendy lineárnych riešičov (MUMPS, HSL MA57/MA97) sú porovnané na reprezentatívnych okruhoch a je analyzovaná parametrická citlivosť optimálneho času na kolo.

Klíčová slova: NLP, simulace času na kolo, optimální řízení, Ipopt, vozidlová dynamika, kolokační metody, Julia, citlivostní analýza

Překlad názvu: Metody numerického optimálního řízení pro minimalizaci času na kolo

Contents

1 Introduction	1
1.1 Motivation	1
1.2 Goals	1
1.3 Structure of thesis	2
1.4 Literature overview/ survey of lap-time simulators	2

Part I

Theory

2 Vehicle and Track modeling framework	7
2.1 Graph centered approach for vehicle modeling	7
2.2 Tire - road interface modeling	7
2.2.1 Linear tire	8
2.2.2 Magic formula tire	9
2.3 Chassis model	9
2.4 Wheel Pivot model	10
2.5 Aerodynamics model	11
2.6 Suspension model	11
2.6.1 Static suspension	12
2.6.2 Dynamic suspension	12
2.7 Gearbox model	13
2.8 Accumulator model	13

2.9 Motor model	14
2.10 Brake model	14
2.11 Car models	15
2.11.1 Generic car model structure	15
2.11.2 Mass Point Quasi Steady State model	16
2.11.3 Single-track	16
2.11.4 Twin-track model	17
2.12 Track modeling	18
2.13 Change of independent variable, time to path	19
3 The Optimal Control Problem	21
3.1 Minimum maneuvering time problem as initial value problem .	21
3.1.1 Quasi Steady state lap time simulation	21
3.2 Transcription methods	23
3.2.1 Shooting methods	24
3.2.2 Nonlinear Programming with Runge Kutta constraints	24
3.2.3 Collocation methods	25
3.3 Collocation at Legendre-Gauss points	27
3.4 Collocation at Legendre-Gauss-Radau points	28
3.5 Collocation at Legendre-Gauss-Lobatto points ...	28
3.5.1 h and p methods for collocation	29
3.6 Mesh refinement methods	29
3.6.1 P adaptive methods	30
3.6.2 h adaptive methods	30
3.6.3 hp adaptive methods	30
3.7 Adjoint methods	30
4 Solvers for nonlinear programming	31
4.1 Nonlinear programming	32
4.1.1 KKT conditions	32
4.2 Interior point methods: IPOPT, MadNLP	33
4.2.1 Barrier problem formulation .	33
4.3 Sequential quadratic programming	34
4.4 Calculation of derivatives	34
4.5 Scaling and matrix conditioning	34
4.6 Sparsity	35
4.7 Initialization	35

4.8 Global and local optimums	35
-------------------------------------	----

Part II Implementation

5 Frameworks for Optimal Control Problem Formulation	39
5.1 CasADi	40
5.2 OpenMDAO + dymos	40
5.3 Acados	40
5.4 GPOPS-II	40
5.5 Julia + JuMP + infititeOpt / examomdels	40
6 My implementation, Julia + JuMP	43
6.1 sensitivity Analysis	44
6.2 Implemented nonlinear programming solvers	45
6.3 Implementation of mesh refinement	46
6.4 Implementation of collocation methods	48
6.5 Implementation of models	49
6.5.1 Tire model implementation ..	49
6.5.2 Implementation of vehicles ..	49
6.6 Benchmark tracks	51
6.7 Example optimization problem .	52

Part III Results

7 Comparison of methods	55
7.1 Benchmark of collocation methods	55
7.2 Benchmark of Linear solvers ...	56
7.3 Benchmark of NLP solvers	58
7.4 Benchmark of derivatives computation	59
8 Conclusion	61

Appendices

A Bibliography	65
-----------------------	-----------

Figures

2.1	Wheel Pivot	7	6.1	Optimization software stack . . .	44
2.2	Tire component	8	6.2	Node-interval ODE error for power-test case.	45
2.3	Wheel Pivot component	10	6.3	NLP solution flow with the methods used in this thesis highlighted in green	46
2.4	Aerodynamics component	11	6.4	Mesh refinement layer with the strategies implemented in this thesis highlighted in green.	46
2.5	Suspension component	12	6.5	Node-interval ODE error for power-test case.	47
2.6	Gearbox component	13	6.6	Discretization error before mesh refinement	47
2.7	Motor component: electrical power P_{elec} enters from the battery side, shaft angular velocity ω is imposed by the drivetrain, output is shaft torque T	14	6.7	Sampling density after mesh refinement	48
2.8	Generic car model structure, which allows all vehicle parameters to be controlled and optimized over.	16	6.8	Transcription layer with the methods used in this thesis highlighted in green	48
2.9	Single-track (bicycle) model . . .	17	6.9	Single-track and Formula Student twintrack vehicle models.	50
2.10	Twin-track model	18	6.10	Formula E and Bus vehicle models.	51
2.11	Generic car model wrapped in a time-to-path transformation layer	19	6.11	Formula Student benchmark tracks.	51
3.1	Test track used for the forward/backward pass example.	22	6.12	Berlin Formula E circuit and the DoubleTurn synthetic track.	52
3.2	Speed profile from forward and backward passes with Quasi Steady state model	22	6.13	FigureEight synthetic track.	52
3.3	Tree of approaches to continuous-time optimal control, adapted from [33], with the direct-methods branch expanded.	24			
3.4	Collocation transcription methods	26			
3.5	Schematic Showing the Differences Between LGL, LGR, and LG Collocation Points, taken from [29]	27			
3.6	Mesh refinement categories [23, 58].	30			
4.1	Unifying framework: the “wheel of strategies”[70]	31			
4.2	NLP solution flow	32			
5.1	Optimal control problem solving pipeline. Adapted from [15, p. 91]	39			

Chapter 1

Introduction

1.1 Motivation

Topic of this thesis is motivated, by my work in Formula Student team eForce Prague Formula, where I worked on real-time vehicle dynamics control software. There I got intricately exposed to problems faced by racing teams, such as should we increase the aerodynamic downforce at the cost of higher drag? Should we put accumulator in the middle of vehicle, to decrease moment of inertia? Should the aerodynamics be designed for straight line drive, or for steady state turning? How to use limited energy from accumulator more effectively, how faster can we go if we made better power limiting algorithm? How large the accumulator should be? Which torque allocation algorithm should be used? How to design the suspension kinematics to maximize the tire utilization? Is rear wheel steering worth the increased weight? All of these problems can be analyzed by using purpose built tool, good engineering practices or using pen and paper with heavily simplified model. Formula student season is 10 months for development and construction of the vehicle, 2 months for competition. That puts enormous pressure on deciding on vehicle concept *fast*. In my opinion what formula student teams need is an open source tool to quickly evaluate different car concepts. As my bachelor thesis I modified existing code into a prototype of such tool, goals of this thesis come from computational and user experience from my previous tool.

1.2 Goals

- Primary goal of this thesis is to investigate available methods for numerical optimal control with respect to their suitability for planning an optimal trajectory of a vehicle aiming to achieve the minimum lap time on a racing circuit.
- Formulate the minimum lap time problem as an optimal control problem.
- Conduct a brief literature review of fundamental methods for numerical optimal control, focus primarily on direct transcription methods

- Implement and benchmark performance of available numerical methods
- The implementation should promote scalability of vehicle model, transcribing and solving methods
- Make the code and thesis available on github <https://github.com/ce-serik/SLapSim.jl>

1.3 Structure of thesis

First part of thesis is vehicle modeling in the context of optimal control. A universal vehicle and track model structure inspired by graph theory from [16] is presented. Following this, various methods of transcribing the problem into nonlinear programming or solving the problem are introduced. The subsequent section describes the basic working principles of Lagrange-Newton solvers of nonlinear programming. After that, frameworks for creating and solving nonlinear programming problems. Finally, my implementation using one framework with benchmark of different vehicle models, transcription methods and solvers follows.

1.4 Literature overview/ survey of lap-time simulators

Lap-time simulation dates back to the 1950s[53]. Computers became common in the 1970s for lap-time simulation and quasi-steady-state (QSS) methods became the standard approach for minimum lap time simulation. QSS is still what most race engineers mean by “lap-time simulation”. In QSS the vehicle is a point mass on a fixed racing line, with lateral and longitudinal accelerations bounded by a steady-state performance envelope [62]. Transient effects such as yaw acceleration are ignored. Minimum lap time simulation was first formulated as an optimal control problem in the late 1980s [35]. A survey of current methods is given in [53]. The work of Perantoni and Limebeer [60] is the main reference: it uses a track-fixed coordinate system, switches the independent variable from time to traveled distance, and applies the formulation to a twin-track Formula 1 model with some vehicle parameters as decision variables. Parts of this thesis build on it. Minimum lap time simulation is still an active research area because better predictions give measurable setup gains, but higher fidelity models cost more solver time. The authors of [2] use sequential convex programming to cut compute cost without losing accuracy.

A very popular lap-time simulator is OptimumLap from OptimumG [3], It is a free tool advertised to achieve 10% accuracy with real vehicles. It uses Quasi Steady State methods (QSS), this is the standard tool for lap-time simulations. Open source implementation of the same method [54], My implementation of this method in chapter ?? . QSS methods are very fast, but the models for which they can be used is equally very simple.

Popular open source C++ implementation of laptime simulator using nonlinear programming and automatic differentiaotn[52]. It uses IPOPT and CppAD[7]. Vehicle model is 6DOF twintrack. Other implementations are work from austria or germany[37] class project from Georgia institute of technology [1]



Part I

Theory

Chapter 2

Vehicle and Track modeling framework

Goal of the thesis is to select a sufficiently accurate yet computationally tractable model of the vehicle. There is not one model of vehicle that would fit all requirements of race engineers. For each problem, a different complexity of the vehicle model is sufficient, e.g. for analysis of power limit algorithm, the lateral dynamics of the vehicle can be heavily simplified and simple transcription and solving methods yield sufficient accuracy within a fraction of a second, this is described in 3.1.1. Whereas analysis of torque allocation problem requires the full complex twintrack model and more computational expensive transcription and solving methods. Therefore, a vehicle modeling framework is needed to satisfy the needs of different types of analyses.

2.1 Graph centered approach for vehicle modeling

The structure of the vehicle model has to allow extending/changing components, so that different vehicle concepts can be simulated, with minimal coding. Book in [16] describes modeling framework that allows creating car components as modules with flow and effort interface. This common interface then heavily simplifies process of creating a car model, as modules can be reused and chained together easily. Vehicle components are chassis, wheel pivot, tire, aerodynamics, suspension, motor and accumulator.

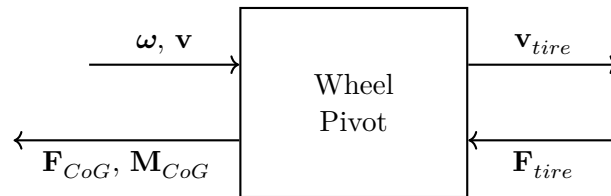


Figure 2.1: Wheel Pivot

2.2 Tire - road interface modeling

Job of the tire model in this thesis is to calculate forces acting on tire from the relative velocity and angle between tire and road. Input of general tire

model is 3 speeds and 3 angles, output is 3 forces.

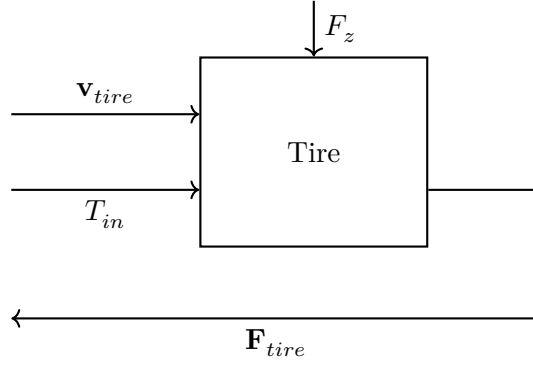


Figure 2.2: Tire component

Most forces that govern vehicle movement are transferred by the tires. The tire is considered to be the most important system of a vehicle and it is notoriously difficult to model [55]. Thus tires need to be modeled with great caution in mind. Not only are there many parameters, such as tire wear, tire temperature, road roughness, road wetness, tire pressure, etc., that affect the resultant forces, they are also constantly changing during the lap. Therefore they are difficult to measure and model precisely. TOTO SOM SKOPIROVAL Z BAKALRKY, V CLANKOCH SOM VIDEL DOSLOVNE KOPIROVANIE SEBA, MOZEM TO ROBIT JA? Implementation of specific models is conver in part III Implementation. [19].

2.2.1 Linear tire

This tire model maps lateral slip angle

$$\alpha = -\arctan\left(\frac{v_y}{v_x}\right), \quad (2.1)$$

into lateral force

$$F_y = \frac{\alpha}{\alpha_{\max}} \mu F_z. \quad (2.2)$$

Tire model does not consider spinning of the wheels, therefore there is no longitudinal slip and motor force acts directly on the wheel pivot point

$$F_x = \frac{T_{\text{wheel}}}{R} + F_{\text{brake}}. \quad (2.3)$$

Forces are constrained by friction ellipse

$$F_x^2 + F_y^2 \leq (\mu F_z)^2. \quad (2.4)$$

Implication of neglecting tire spinning is that, energy from motor is not used to spin up the wheels and all of it is used to accelerate the car, creating large discrepancy from how a real vehicle operates. In this and following models this effect is not accounted for.

2.2.2 Magic formula tire

Magic formula tire developed by Hans B. Pacejka [36] is an empirically derived formula, that describes tire forces. Full magic formula model has around 80 parameters and many variants. Variant described here was taken from [1] and has only 6 parameters.

$$\alpha = -\arctan\left(\frac{v_y}{v_x}\right). \quad (2.5)$$

Parameter D is peak factor

$$D = \mu F_z \left(1 - \epsilon \frac{F_z}{F_{z,\text{ref}}}\right), \quad (2.6)$$

ϵ models degressive behavior, the peak lateral force of real tire does not grow linearly with normal force. $F_{z,\text{ref}}$ is nominal normal tire load [1]. Lateral force is then computed according to magic formula from [36],

$$F_y = D \sin\left(C \arctan(B\alpha - E(B\alpha - \arctan(B\alpha)))\right), \quad (2.7)$$

Where parameters C is shape factor, B is stiffness factor, E is curvature factor. These parameters are fitted to represent real tire measurement data. Then tire rolling resistance is added C_r and the friction ellipse is the same as linear tire,

$$F_x = \frac{T_{\text{wheel}}}{R} + F_{\text{brake}} + C_r F_z, \quad (2.8)$$

$$\left(\frac{F_x}{\mu F_z}\right)^2 + \left(\frac{F_y}{\mu F_z}\right)^2 \leq 1. \quad (2.9)$$

2.3 Chassis model

The chassis is modeled as a point mass located at the center of gravity (CoG), carrying the full vehicle mass and yaw inertia, with the wheel pivot points defined relative to it. The chassis model is responsible for computing the state-derivative vector from all forces and moments acting on the CoG. Equations 2.10-2.13 describe the computation.

Table 2.1: Parameters of the chassis model.

Symbol	Unit	Description
m	kg	Mass
I_z	$kg \cdot m^2$	Yaw Inertia
w	m	width of vehicle

$$\mathbf{M}_{CoG,i} = \mathbf{r}_i \times \mathbf{F}_i, \quad \mathbf{F}_i = \begin{bmatrix} F_{x_i} \\ F_{y_i} \\ 0 \end{bmatrix}. \quad (2.10)$$

$$\mathbf{F}_{CoG} = \sum_i \mathbf{F}_i + \begin{bmatrix} F_{\text{drag}} \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{M}_{CoG} = \sum_i \mathbf{M}_{CoG,i}. \quad (2.11)$$

$$\dot{\mathbf{v}} = \frac{\mathbf{F}_{CoG}}{m} - \boldsymbol{\omega} \times \mathbf{v}, \quad \dot{\boldsymbol{\omega}} = \frac{\mathbf{M}_{CoG}}{I}. \quad (2.12)$$

$$\mathbf{x} = \begin{bmatrix} v_x \\ v_y \\ \psi \\ \dot{\psi} \end{bmatrix}, \quad \dot{\mathbf{x}} = \begin{bmatrix} \dot{v}_x \\ \dot{v}_y \\ \dot{\psi} \\ \ddot{\psi} \end{bmatrix}. \quad (2.13)$$

Chassis also imposes constraint on location on track to prevent the vehicle leaving the track.

2.4 Wheel Pivot model

Wheel pivot point is modeled as transformer. It defines the transformation of CoG velocities into tire frame and then transforms tire forces and torque into pivot frame. Angular velocity and position vectors are formed from yaw rate $\dot{\psi}$ and the longitudinal distance l_i of the axle from the center of gravity:

$$\boldsymbol{\omega} = \begin{bmatrix} 0 \\ 0 \\ \dot{\psi} \end{bmatrix}, \quad \mathbf{r}_i = \begin{bmatrix} l_i \\ b/2 \\ 0 \end{bmatrix}, \quad (2.14)$$

where b is the track width. Velocity at each tire pivot point is:

$$\mathbf{v}_{p_i} = \mathbf{v} + \boldsymbol{\omega} \times \mathbf{r}_i. \quad (2.15)$$

Velocities are then rotated from the pivot frame into the tire frame using the steering angle δ_i :

$$R_t = \begin{bmatrix} \cos \delta_i & -\sin \delta_i & 0 \\ \sin \delta_i & \cos \delta_i & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{v}_{t_i} = R_t \mathbf{v}_{p_i}. \quad (2.16)$$

Tire forces F_{tx} and F_{ty} are computed from $\mathbf{v}_{t_i}$ by the tire model. Forces are then rotated back to the pivot frame:

$$\begin{bmatrix} F_{x_i} \\ F_{y_i} \end{bmatrix} = \begin{bmatrix} \cos \delta_i & \sin \delta_i \\ -\sin \delta_i & \cos \delta_i \end{bmatrix} \begin{bmatrix} F_{tx} \\ F_{ty} \end{bmatrix}. \quad (2.17)$$

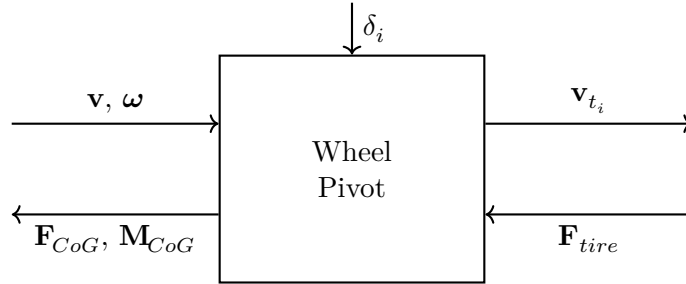


Figure 2.3: Wheel Pivot component

2.5 Aerodynamics model

Aerodynamic model is modeled as nonlinear resistance element, mapping velocity into forces. Drag and down-force scale with the square of longitudinal velocity:

$$F_{\text{drag}} = \frac{1}{2} \rho C_D v_x^2, \quad F_{\text{lift}} = \frac{1}{2} \rho C_L v_x^2. \quad (2.18)$$

The down-force is split between front and rear axles by the center of pressure ratio $\text{CoP} \in [0, 1]$, where $\text{CoP} = 0.5$ corresponds to a balanced distribution. Air density ρ is taken as 1.225 kg/m^3 at sea level. Coefficients C_L , C_D and CoP used per vehicle model are listed in Table 2.2.

Table 2.2: Parameters of the aerodynamic model.

Symbol	Unit	Description
C_L	—	Lift coefficient (negative = down-force)
C_D	—	Drag coefficient
CoP	—	Center of pressure (front axle share, 0–1)
ρ	kg/m^3	Air density

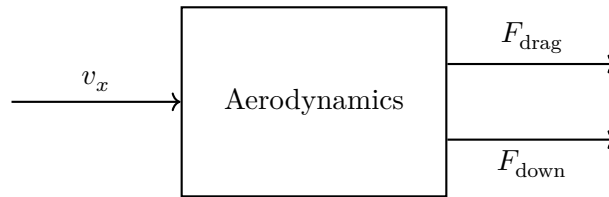


Figure 2.4: Aerodynamics component

This model is sufficient when vehicle is driving relatively straight, that means lateral acceleration is roughly zero. With high lateral acceleration this model is no longer valid, because air speeds are different on left and right part of car and also the car starts rolling- one side of vehicle is closer than the other, creating imbalance of lift forces.

2.6 Suspension model

Purpose of suspension in car is to maintain tire contact, distribute the load from lateral load transfer to better utilize tires and use kinematics to make use of camber thrust. Commonly suspension of car consists of spring, damper and the mounting with tie rods. In this thesis Suspension model had to be significantly simplified. Modeling each wheel with a damper, spring and its own kinematics is beyond the scope of this thesis. However, the model structure allows for it, I didn't have enough time to implement it. This quarter car model would be very beneficial toward the goal of scalability. Without it, the model scalability is limited, as dividing force between more than two wheels on an axis is a statically indeterminate problem.

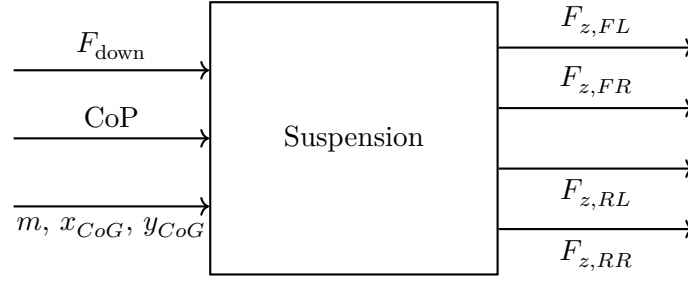


Figure 2.5: Suspension component

Two suspension variants are used in this thesis, static distribution and dynamic distribution

2.6.1 Static suspension

Static suspensions distributes the vehicle's mass and force from aerodynamics according to position of center of gravity and center of pressure. This trivial implementation is possible only for 2 axles, having 3 and more axles would make this a statically indeterminate system.

$$\begin{aligned}
 F_{z,FL} &= (mg x_{CoG} + F_{lift} CoP)(1 - y_{CoG}) \\
 F_{z,FR} &= (mg x_{CoG} + F_{lift} CoP) y_{CoG} \\
 F_{z,RL} &= (mg(1 - x_{CoG}) + F_{lift}(1 - CoP))(1 - y_{CoG}) \\
 F_{z,RR} &= (mg(1 - x_{CoG}) + F_{lift}(1 - CoP)) y_{CoG}
 \end{aligned} \tag{2.19}$$

2.6.2 Dynamic suspension

This suspension variant takes into account the load transfer caused by longitudinal and lateral accelerations. It is taken from [1].

$$\begin{aligned}
 F_{z,FL} &= \left(mg x_{CoG} - \frac{m a_x h_{CoG}}{L} \right) (1 - y_{CoG}) + \frac{1}{2} F_{down} CoP - \frac{1}{2} \Delta F_y \\
 F_{z,FR} &= \left(mg x_{CoG} - \frac{m a_x h_{CoG}}{L} \right) y_{CoG} + \frac{1}{2} F_{down} CoP + \frac{1}{2} \Delta F_y \\
 F_{z,RL} &= \left(mg(1 - x_{CoG}) + \frac{m a_x h_{CoG}}{L} \right) (1 - y_{CoG}) + \frac{1}{2} F_{down} (1 - CoP) - \frac{1}{2} \Delta F_y \\
 F_{z,RR} &= \left(mg(1 - x_{CoG}) + \frac{m a_x h_{CoG}}{L} \right) y_{CoG} + \frac{1}{2} F_{down} (1 - CoP) + \frac{1}{2} \Delta F_y
 \end{aligned} \tag{2.20}$$

ma_x is the estimate of longitudinal acceleration taken from the sum of driving, braking and rolling resistance forces.

$$ma_x \approx F_{drive} + F_{brake} - F_{drag} - C_r mg. \tag{2.21}$$

ΔF_y is a new decision variable that describes lateral load transfer. It is subject to constraint

$$\Delta F_y = \frac{h_{\text{CoG}}}{t} \sum_4 F_{y,i}, \quad (2.22)$$

where $F_{y,i}$ are lateral forces at pivot points of wheels. This is a Quasi steady state model of suspension, as the load transfer is considered to be instant, in real vehicle a system of spring and damper on each wheel introduces more complex dynamics.

2.7 Gearbox model

Gearbox is the textbook example of transformer and resistive element.

$$T_{\text{out}} = \eta i T_{\text{in}}, \quad \omega_{\text{out}} = \frac{\omega_{\text{in}}}{i}. \quad (2.23)$$

Table 2.3: Parameters of the gearbox model.

Symbol	Unit	Description
i	—	Gear ratio (input to output speed)
η	—	Mechanical efficiency

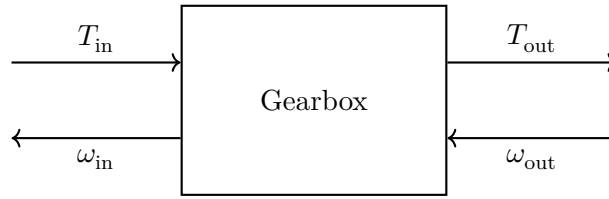


Figure 2.6: Gearbox component

2.8 Accumulator model

Accumulator model sums power from motors

$$P_{\text{acc}}(t) = \sum_{i=1}^{N_m} P_{\text{el},i}(t), \quad (2.24)$$

then constraints the power according to accumulator limits

$$P_{\text{acc}}^{\min} \leq P_{\text{acc}}(t) \leq P_{\text{acc}}^{\max}. \quad (2.25)$$

What is missing in this thesis, is a total energy constraint for a lap.

2.9 Motor model

Motor is modeled as an effort source with efficiency. The efficiency is traditionally modeled using *max* function in 2.27, this function is not differentiable.

$$P_{\text{mech}} = T \omega, \quad P_{\text{elec}} = \max\left(\frac{P_{\text{mech}}}{\eta}, P_{\text{mech}} \eta\right). \quad (2.26)$$

Max function cannot be used in nonlinear programming, different formulation has to be used. This thesis uses constraints

$$\begin{aligned} P_{\text{elec}} &\geq \frac{P_{\text{mech}}}{\eta}, \\ P_{\text{elec}} &\geq P_{\text{mech}} \eta. \end{aligned} \quad (2.27)$$

Problem with this formulation is that, when full power is not required, that is P_{acc} is strictly inside constraints

$$P_{\text{acc}}^{\min} \leq P_{\text{acc}}(t) \leq P_{\text{acc}}^{\max}, \quad (2.28)$$

the power draw from accumulator may be higher compared to the classical formulation with *max* function. Another possible formulation is using smooth max approximation using arctan or $\sqrt{\cdot}$ or softmax function, I have covered this approach in bachelor thesis [19].

Table 2.4: Parameters of the motor model.

Symbol	Unit	Description
T	Nm	Shaft torque
η	—	Motor + inverter efficiency
P_{mech}	W	Mechanical shaft power
P_{elec}	W	Electrical port power (battery side)

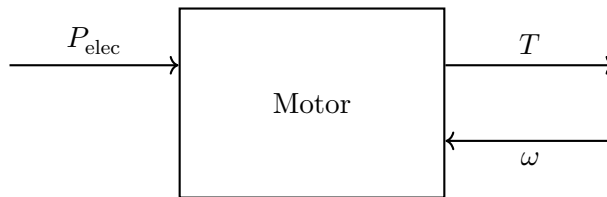


Figure 2.7: Motor component: electrical power P_{elec} enters from the battery side, shaft angular velocity ω is imposed by the drivetrain, output is shaft torque T .

2.10 Brake model

Braking force is distributed among wheels by brake bias ratio, to exclude simultaneous braking and accelerating a complementarity constraint is used,

$$T_m \cdot F_b \leq \varepsilon, \quad T_m \geq 0, \quad F_b \geq 0, \quad (2.29)$$

Where T_m is driving torque or sum of driving torques, F_B is braking torque. A cleaner alternative is to penalize energy consumption in the objective, then the explicit complementarity constraint is not required.

2.11 Car models

These models must work when used in NLP transcription for solving boundary value problem, but they also must work for initial value problem, when the model is initialized or verified for transcription error. Some toolboxes can automatically create constraints from ODE models [74]. In my implementation I have an if statement that switches between IVP and BVP variant of models equations, major disadvantage of this approach is that the equations for both variants must be effectively the same and finding bugs is difficult with this approach.

2.11.1 Generic car model structure

Leveraging the full potential of automatic differentiation methods, any car Parameter can be assigned as state, control, static parameter, decision variable or sensitivity analysis variable. This allows for changing a fixed parameter into a degree of freedom allowing to decide which parameters should be controlled. This also allows for finding the optimal parameters of vehicle, such as brake bias, aerodynamics balance, gear ratio during lap time optimization. This is shown on figure 2.8. When parameter is chosen to be sensitivity analysis variable, impact of changing this variable is cheaply computed after the optimization is finished, see Section 3.7. Each parameter holds three values: lower bound, upper bound, and nominal value. The bounds are passed directly to the NLP as variable limits and are reused to scale the variable. Constraint that wheels cannot leave the track is created using upper and lower bound distance from centerline, which is a state of the car.

All vehicles must have at least these states:

- v_x — longitudinal velocity in the body frame [m/s],
- v_y — lateral velocity in the body frame [m/s],
- ψ — heading angle relative to the track tangent [rad],
- $\dot{\psi}$ — yaw rate [rad/s],
- n — signed lateral distance from the centerline [m],
- t — elapsed lap time [s].

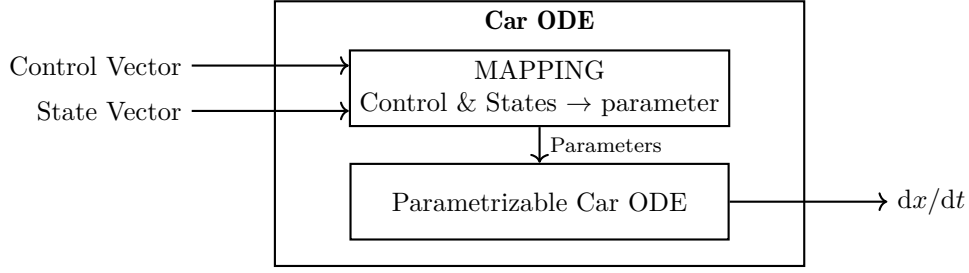


Figure 2.8: Generic car model structure, which allows all vehicle parameters to be controlled and optimized over.

■ 2.11.2 Mass Point Quasi Steady State model

Simple and useful car model can be made using these equations

$$F_z = \frac{1}{2} \rho C_L v_x^2, \quad (2.30)$$

$$F_y = m v_x^2 c, \quad (2.31)$$

$$F_{x,\text{pwr}} = \frac{P_{\max}}{v_x}, \quad (2.32)$$

$$F_{x,\max}^2 = \max\left(\left[(F_z + mg)\mu\right]^2 - F_y^2, 0\right), \quad (2.33)$$

$$F_x^2 + F_y^2 \leq \left[(F_z + mg)\mu\right]^2, \quad (2.34)$$

$$|F_x| \leq \min(F_{\text{mot},\max}, F_{x,\text{pwr}}), \quad (2.35)$$

$$\dot{v}_x = \frac{F_x - \frac{1}{2} \rho C_D v_x^2}{m}. \quad (2.36)$$

Where ρ is air density C_L is lift coefficient v_x is longitudinal speed, m is mass, c is curvature of track in current point, $P_{\max}$ is maximum power of motor, μ is friction coefficient, C_D is drag coefficient. This model has only one state v_x which makes it possible to use specialized lap time simulation solvers. Variations of this model, where for example $P_{\max}$ is a lookup table, the grip does not scale linearly with down-force etc. are commonly used in motor-sport.

■ 2.11.3 Single-track

A more complex and useful model is single-track. In this model the suspension effects are neglected. Single-track is still considered to be simplified model and is used most commonly when more complex models are too computationally expensive. A computational graph of how this model is assembled from the defined components is shown in Figure 2.9.

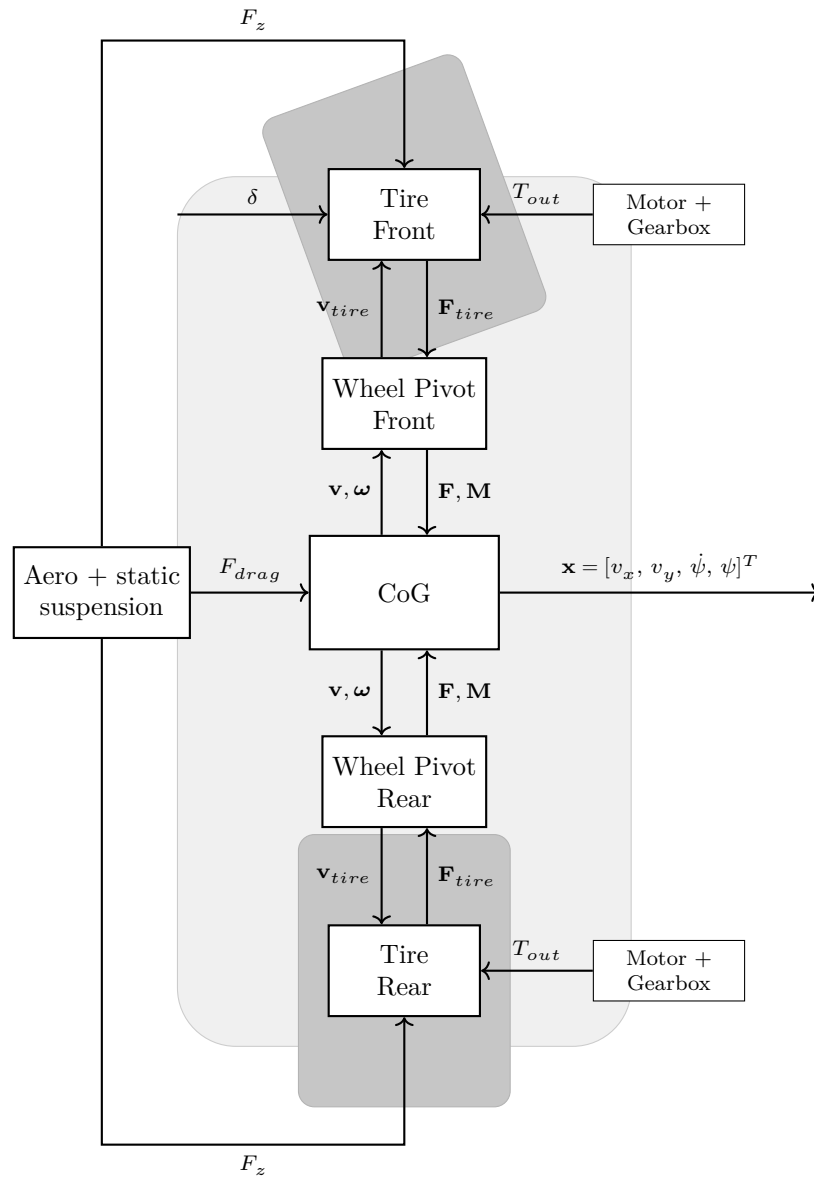


Figure 2.9: Single-track (bicycle) model

2.11.4 Twin-track model

Twin-track model has all four tires modeled separately. This captures left-right load transfer. Computational graph of it is shown in figure 2.10

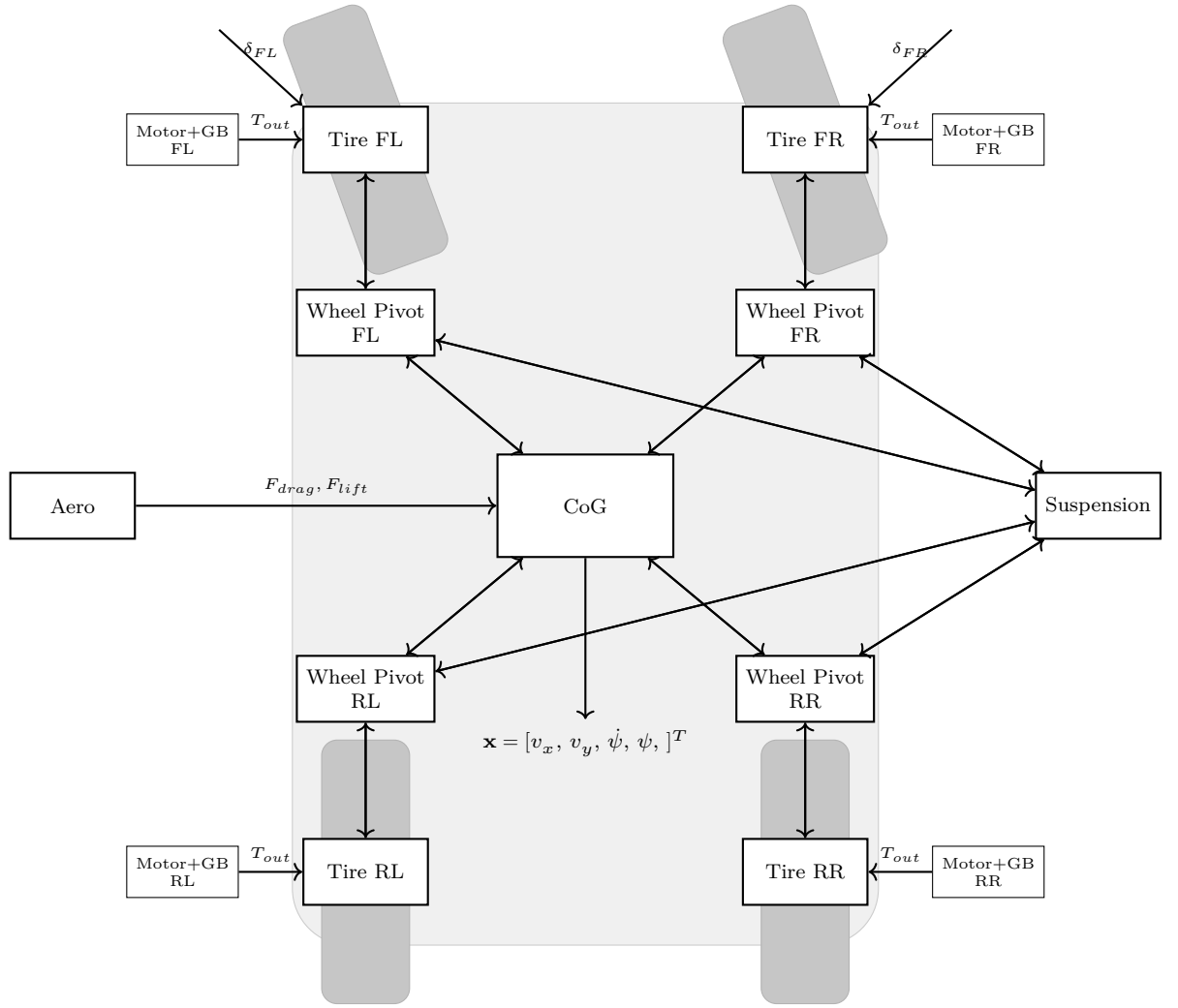


Figure 2.10: Twin-track model

2.12 Track modeling

Track is sampled at discrete arc length nodes s_i and stored as a set of fields evaluated at each node:

- centerline coordinates $x(s)$, $y(s)$,
- heading $\theta(s)$,
- curvature $\kappa(s)$,
- left and right track widths $w_L(s)$, $w_R(s)$,
- air density $\rho(s)$,
- friction coefficient $\mu(s)$,

- inclination $\phi(s)$.

Each field is linearly interpolated between the segments. Obtaining suitable track for optimization from GPS is non-trivial, simple methods lead to noisy data, which then corrupts the optimization result. Therefore, smoothing method described in [60] has been used.

2.13 Change of independent variable, time to path

When differential equations for vehicle movement are stated. For use in lap time simulation it is beneficial to describe the car states not in time but in distance traveled. This introduces connection with location on track and reduces number of state variables by one. This transformation is taken from [60].

$$\varepsilon = \psi - \theta(s) \quad (2.37)$$

where ψ is car heading, $\theta(s)$ is heading of track at distance s ,

$$S_f = \frac{dt}{ds} = \frac{1 - nC(s)}{v_x \cos \varepsilon - v_y \sin \varepsilon}, \quad (2.38)$$

n is vehicles distance from centerline, $C(s)$ is curvature of track at distance s . The change in n is defined as:

$$\frac{dn}{dt} = v_x \sin \varepsilon + v_y \cos \varepsilon. \quad (2.39)$$

Then the model can be transformed from time as independent variable to distance travelled along centerlines as independent variable by equation:

$$\frac{d}{ds} \begin{pmatrix} v_x \\ v_y \\ \psi \\ \dot{\psi} \\ n \\ t \end{pmatrix} = S_f \begin{pmatrix} \dot{v}_x \\ \dot{v}_y \\ \dot{\psi} \\ \ddot{\psi} \\ \dot{n} \\ 1 \end{pmatrix}. \quad (2.40)$$

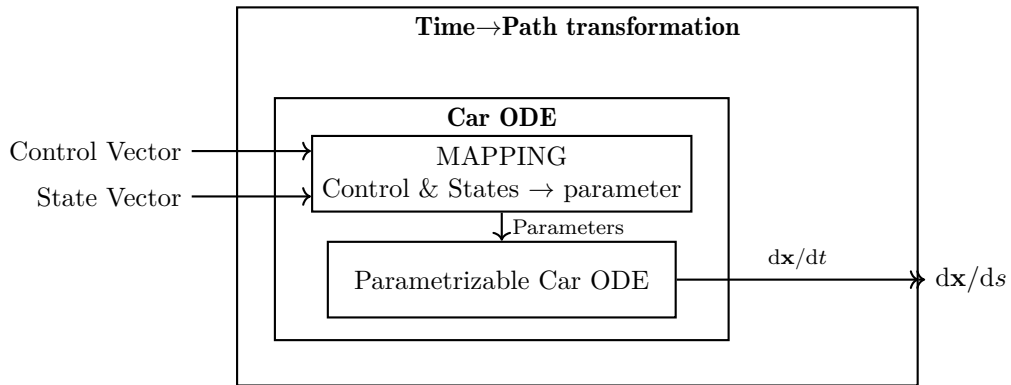


Figure 2.11: Generic car model wrapped in a time-to-path transformation layer

2. *Vehicle and Track modeling framework*



Now the vehicle is modeled and placed on the track. Next step is to drive it as fast as possible.

Chapter 3

The Optimal Control Problem

In previous chapter I have described how to build differential equations that govern movement of a vehicle. This chapter describes two approaches of driving that vehicle optimally, first by using classical initial value problem/time marching formulation, then reformulating it to boundary value problem.

3.1 Minimum maneuvering time problem as initial value problem

The most common method for solving an ODE is as initial value problem. An industry standard high fidelity multi-body time marching solver for vehicles is Car Maker from IPG [39]. It is great for driving simulation, testing and development of advanced driver assistant systems, modeling the vehicles and racetrack modeling, it can be used for lap-time simulation, but with caveats. In time marching solvers a separate controller is required, which becomes part of the ODE. This approach gives a solution to the problem of how to control car, so that it completes a lap around a circuit. However, this solution is almost certainly not optimal to time or any meaningful objective. Another issue is that if the vehicle's parameters change and the controller does not, the resulting time can be exactly same, because controller cannot take advantage of different vehicle's properties. Sensitivity analysis and lap time can be obtained but it is skewed by the chosen controller. The controller can be tuned for specific vehicle, but that will still not guarantee optimal control. Using time marching is not the best for sensitivity analysis and lap time simulation of a complex model.

When the vehicles model is simplified to Quasi Steady State model, time marching simulation can be used.

3.1.1 Quasi Steady state lap time simulation

Following section describes commonly used algorithm which uses only initial value solvers, to compute the minimum maneuvering time. For quasi steady state model 2.36 it is possible to compute optimal speed profile ked dal som rovanke rovnice do nlp a dostal som rovnaky vysledok ako z toho algoritmu, takže aspon pre tuto trat to asi bude optimum speed profile using an algorithm

well known among racing engineers. Figure 3.2 shows the speed profile obtained from this algorithm, on track from Figure 3.1, start is dot and cross is finish.

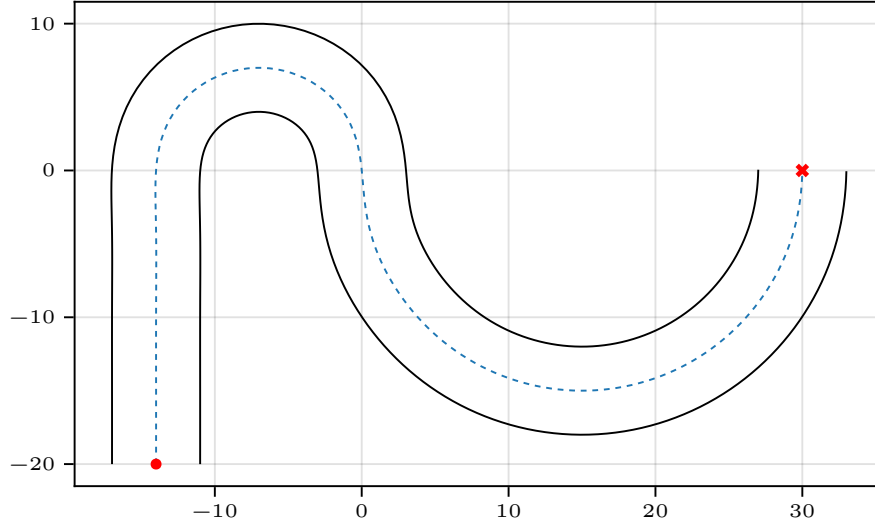


Figure 3.1: Test track used for the forward/backward pass example.

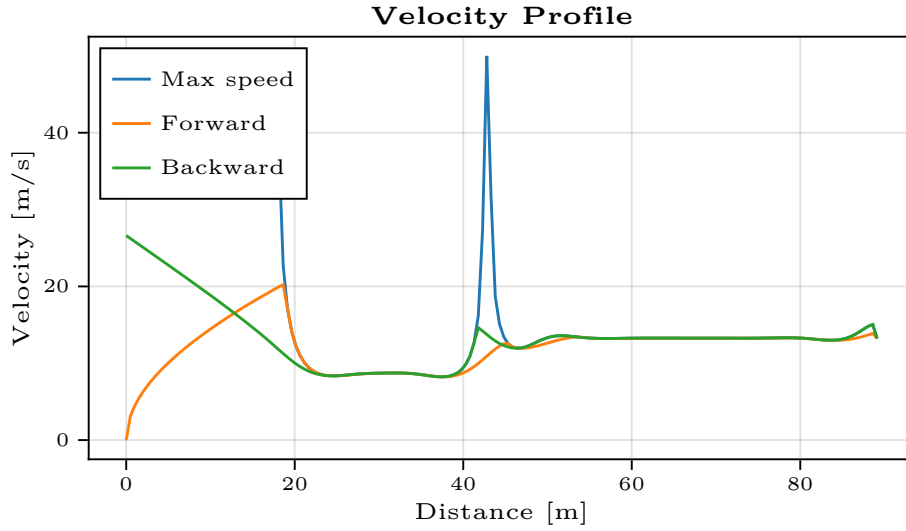


Figure 3.2: Speed profile from forward and backward passes with Quasi Steady state model

Algorithm works in following manner:

1. Maximum speed Pass: From the vehicle model and curvature along track, compute the maximum speed, blue line on Figure 3.2

2. Forward pass: consider only forward acceleration limits of the vehicle (braking force is unlimited), run the vehicle from start to finish along the trajectory, applying maximum longitudinal acceleration and respecting max speed limit, orange line on Figure 3.2
3. Consider only braking acceleration limits of the vehicle (acceleration force is unlimited), run the vehicle *backwards, from finish to start* along the trajectory, applying maximum braking acceleration (braking now increases speed as the simulation is flipped) and respecting max speed limit
4. Take minimum speeds of forward and backward pass and this gives the minimum time speed profile

More complete description of this algorithm is in [18] This is the classic approach to minimum maneuvering time problem, the issue is that the trajectory of vehicle on track is fixed and cannot be optimized, the vehicle model must be heavily simplified, it does not work with any integral constraints like accumulator energy. However, the computation time can be fraction of a second for the entire lap. Tools using this method have proven to be extremely useful and many different analyses can be performed by leveraging its computational speed, for example varying the friction coefficient at on each corner, a corners importance on lap time can be calculated [57].

3.2 Transcription methods

Previous section described automotive specific methods of solving minimum maneuvering time problem, this section explains methods of transcribing the problem into the language of optimal control problem. Transcription methods rewrite continuous (infinite-dimensional) OCP into finite-dimensional NLP. That means representing the continuous trajectory of a state by values sampled at discrete points. The terminology of transcription methods can be difficult to understand, as it is complex topic and often articles present conflicting information [49]. Website [49] and other work from Matthew Kelly [46, 44, 47] offers a very digestible introduction into the topic. This thesis focuses only on direct methods. Classification of methods in this thesis is based on [49] Figure 3.3 shows how transcription methods are commonly classified.

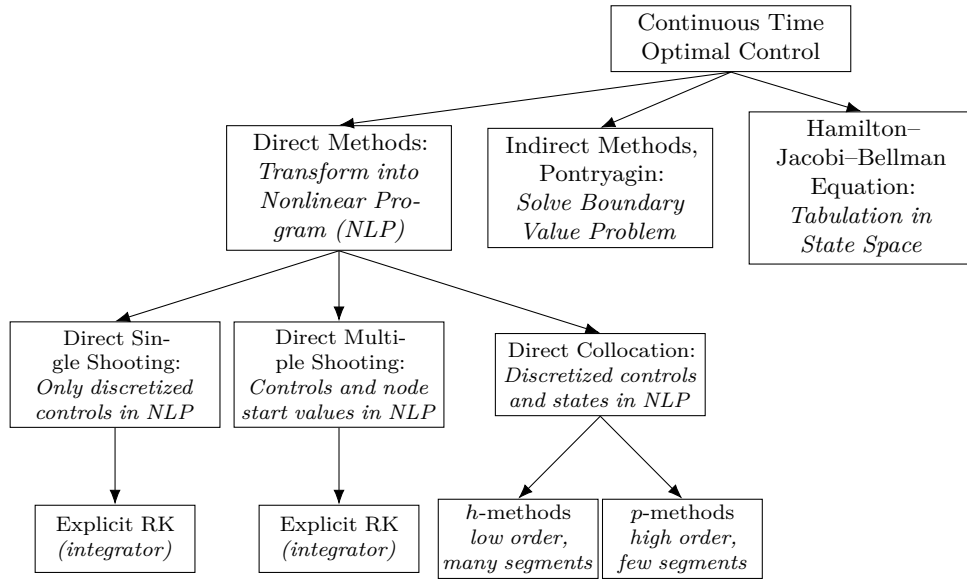


Figure 3.3: Tree of approaches to continuous-time optimal control, adapted from [33], with the direct-methods branch expanded.

3.2.1 Shooting methods

Shooting methods get their name from aiming artillery canon analogy, with given powder load canon and target location find the aiming angle to hit the target. In depth explanation with animation and example code is here [45] This is classical boundary value problem. Methods shoot, miss, adjust aim, shoot again and iterate until target is hit. These methods are solving boundary value problem using an IVP solver. More precisely,

1. Guess the controls
2. Compute the solution using an initial value problem solver
3. Compare the simulated value at the end of interval from 2. with the boundary value(targets location)
4. Based on error update the controls and go to 2.

Shooting methods are split into single and multiple shooting. They differ in way they handle state discretisation. Single shooting discretizes the state only at start and end of interval and computes the trajectory in one shot. Whereas multiple shooting discretizes the states along the trajectory and computes multiple shots, this can be also parallelized on a CPU [28]. Shooting methods are progressively being displaced by collocation methods [61]

3.2.2 Nonlinear Programming with Runge Kutta constraints

These methods are not using initial value solvers to solve boundary value problem, instead they use nonlinear programming. Most straightforward

method to transcribe continuous optimal control problem into non-linear program, is to use explicit Runge Kutta solving methods from initial value problem as equality constraints on the dynamics of a vehicle. Using Euler method, the constraints for the dynamics of vehicle are:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + h \cdot f(\mathbf{x}_k, \mathbf{u}_k), \quad (3.1)$$

where f denotes the vehicle dynamics, h the step size, and $\mathbf{x}$ the state vector. This works, but it may be the worst possible transcription method, due to high error and instability of Euler method [4]. Problems of Euler method encountered when solving initial value problems also translate to nonlinear programming. Implicit RK methods can be used too, In Initial value problems they are used less often, as they require iterative solving. Optimal control problem is Boundary value problem and not initial value problem, therefore:

...for a boundary value problem ...any scheme becomes effectively implicit. Thus, the distinction between explicit and implicit initial value schemes becomes less important in the BVP context.

[12, p. 69]

For boundary value problem it makes no sense to use explicit Runge Kutta methods, as they have inferior stability compared to implicit Runge Kutta methods.[61, p. 4]. Implicit Runge Kutta methods, which are commonly used are from LobattoIIIA family, but any other method can be used. Book [15] uses only these.

3.2.3 Collocation methods

The concept of collocation is old and universal in numerical analysis ...For ordinary differential equations it consists in searching for a polynomial of degree s whose derivative coincides ("co-locates") at s given points with the vectorfield of the differential equation ...[5, p. 224]

The article [46] is an introductory explanation of inner working and implementation of direct collocation methods, theory here is based on it and on [29]. More in depth theory behind these methods was developed by Lloyd N. Trefethen in [68] and implementation in [69]. Collocation methods are a subset of implicit Runge-Kutta methods. The distinguishing property is that the solution of a boundary value problem is approximated by a piecewise polynomial in the independent variable (distance along the centerline). On each interval the polynomial is fixed by requiring it to satisfy the ODE exactly at N chosen points the collocation nodes.

There are numerous ways to construct a collocation method, in optimal control the classic methods are those based on roots Legendre or Chebyshev type I polynomials, Lagrange polynomials [25, 30] and Gauss quadrature rule. In this thesis only Legendre and Gauss quadrature based collocation methods

are covered. Figure 3.4 depicts multiple ways of constructing collocation methods.

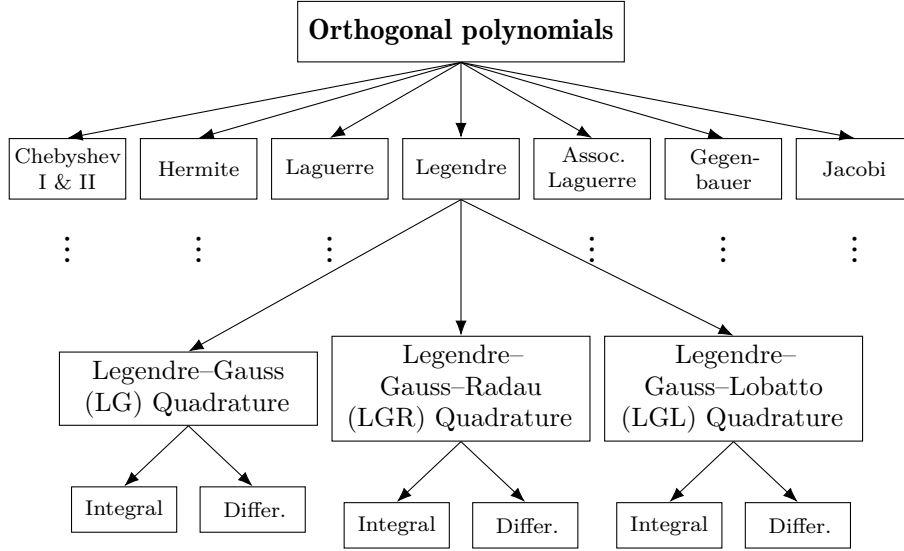


Figure 3.4: Collocation transcription methods

■ Lagrange polynomial

First part of creating a collocation method is creating an interpolating polynomial basis,

$$L_i(\tau) = \prod_{\substack{j=0 \\ j \neq i}}^N \frac{\tau - \tau_j}{\tau_i - \tau_j}, \quad i = 0, \dots, N. \quad (3.2)$$

Where τ is the point to be interpolated and τ_j and τ_i are node positions, in order to get interpolating polynomial $L(\tau)$, values at nodes x_j are multiplied by corresponding Lagrange base and summed

$$L(\tau) = \sum_{i=0}^N f_i L_i(\tau). \quad (3.3)$$

Now it is possible to interpolate a state between the discretized nodes, Note that the node locations for Legendre polynomial are arbitrary. In the actual implementation, calculation of polynomial basis from definition 3.2 is numerically impractical, therefore barycentric form from [13] has been used as implementation of these methods. The next step is to compute definite integral using values from function values at nodes.

■ Gaussian Quadrature rule

Quadrature rule is the approximation of definite integral of a function by a sum of function values at given nodes. In optimal control context Gaussian

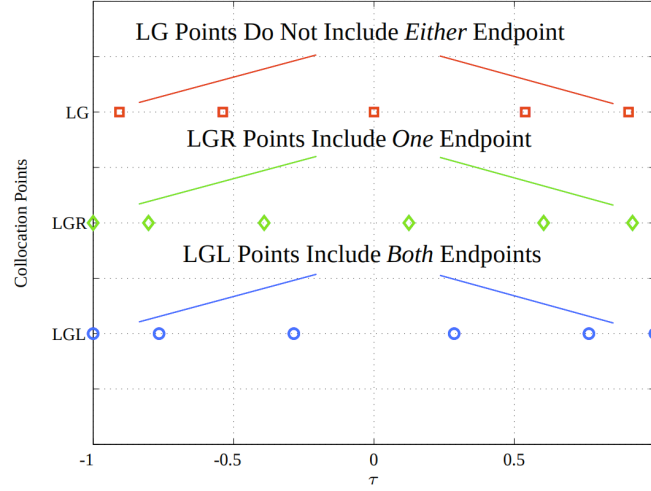


Figure 3.5: Schematic Showing the Differences Between LGL, LGR, and LG Collocation Points, taken from [29]

quadrature is commonly used

$$\int_{-1}^1 f(\tau) d\tau \approx \sum_{i=0}^n w_i f_i, \quad (3.4)$$

where w are Gaussian quadrature weights. A useful property of Gaussian quadrature of order N is that it approximates the integral to machine precision whenever the integrated function is a polynomial of degree at most $2N - 1$. Node selection for Gauss quadrature is *not arbitrary*, in collocation methods the node positions are based on roots of Legendre polynomial. Node positions specify the type of collocation method and are creating in following manner. Let $P_N(\tau)$ denote the Legendre polynomial of degree N . Three node families are commonly used [29]:

- Legendre-Gauss: roots of $P_N(\tau)$, includes no endpoints.
- Legendre-Gauss-Radau: roots of $P_{N-1}(\tau) + P_N(\tau)$, includes one endpoint.
- Legendre-Gauss-Lobatto: roots of $\dot{P}_{N-1}(\tau)$, includes both endpoints.

3.3 Collocation at Legendre-Gauss points

Legendre-Gauss collocation methods uses nodes strictly inside the interval for approximation using polynomial. With N nodes, it is possible to approximate functions up to degree $N - 1$ with machine precision. States are approximated by series

$$x_j^N(\tau) = \sum_{i=0}^N x_{ij} L_i(\tau), \quad (3.5)$$

where j is the number of the state. Then the series is differentiated using barycentric form from [13, p.513] and evaluated at k -th collocation point τ_k

$$\dot{x}_j^N(\tau_k) = \sum_{i=0}^N x_{ij} \dot{L}_i(\tau_k) = \sum_{i=0}^N D_{ki} x_{ij}, \quad D_{ki} = \dot{L}_i(\tau_k). \quad (3.6)$$

Matrix $\mathbf{D}$ is called *Gauss Pseudospectral differentiation matrix*. In matrix notation can be written

$$\mathbf{DX} = \mathbf{F}(\mathbf{X}_{\text{LG}}, \mathbf{U}_{\text{LG}}) \quad (3.7)$$

where $\mathbf{X}_{\text{LG}}$ is matrix of states at Legendre-Gauss node points, $\mathbf{U}_{\text{LG}}$ is matrix of controls at Legendre-Gauss node points, $\mathbf{X}$ is matrix of states at Legendre-Gauss node points and at initial point x_0

This is then used as constraints to formulate a transcription method

$$\begin{aligned} \min \quad & \Phi(\mathbf{X}_{N+1}), \\ \text{s.t.} \quad & \mathbf{DX} = \mathbf{F}(\mathbf{X}_{\text{LG}}, \mathbf{U}_{\text{LG}}), \\ & \mathbf{X}_{N+1} = \mathbf{X}_0 + \mathbf{w}^T \mathbf{F}(\mathbf{X}_{\text{LG}}, \mathbf{U}_{\text{LG}}), \\ & \mathbf{X}_0 = \mathbf{x}_0. \end{aligned} \quad (3.8)$$

Where w are the Gauss quadrature weights. This is the differential form of Legendre-Gauss collocation method, integral form is described in [30].

3.4 Collocation at Legendre-Gauss-Radau points

This method is constructed in similar manner to previous, with difference that one endpoint is a collocation node. This is an advantage, as the resulting nonlinear programming formulation does not require the gauss quadrature rule for endpoint and it is smaller. This method approximates functions up degree $2N - 2$ with machine precision, this is lower precision than Legendre-Gauss.

$$\begin{aligned} \min \quad & \Phi(\mathbf{X}_N) \\ \text{subject to} \quad & \mathbf{DX} = \mathbf{F}(\mathbf{X}^{\text{LGR}}, \mathbf{U}^{\text{LGR}}), \\ & \mathbf{X}_0 = \mathbf{x}_0. \end{aligned} \quad (3.9)$$

This is also a differential formulation.

3.5 Collocation at Legendre-Gauss-Lobatto points

The Gauss-Lobatto family includes both endpoints as collocation nodes and has a quadrature degree of exactness $2N - 3$. It contains classical low-order schemes like the trapezoidal rule and Hermite-Simpson. I implement it using the Butcher tableau of LobattoIIIA implicit Runge-Kutta methods from [24].

The dynamics is collocated at N LobattoIIIA nodes $c_i \in [-1, 1]$ with coefficient matrix a_{ij} taken from the Butcher tableau. $\mathbf{X}_i$ and $\mathbf{U}_i$ are the

state and control at node i , placed at $\tau_i = c_i h/2$, where h is the length of the interval. The node constraints are

$$\mathbf{X}_i = \mathbf{X}_0 + \frac{h}{2} \sum_{j=1}^N a_{ij} \mathbf{F}(\mathbf{X}_j, \mathbf{U}_j), \quad i = 2, \dots, N, \quad (3.10)$$

where $\mathbf{X}_0$ is the left endpoint. The resulting NLP is

$$\begin{aligned} \min \quad & \Phi(\mathbf{X}_N) \\ \text{subject to} \quad & \mathbf{X}_i = \mathbf{X}_0 + \sum_{j=1}^N a_{ij} \mathbf{F}(\mathbf{X}_j, \mathbf{U}_j), \quad i = 2, \dots, N, \\ & \mathbf{X}_0 = \mathbf{x}_0. \end{aligned} \quad (3.11)$$

This is an integral formulation of collocation method, in book [15] trapezoidal and Hermite-Simpson methods written in this form.

3.5.1 h and p methods for collocation

A function can be approximated in three ways. The first uses a single high-order polynomial over the entire trajectory, these are p methods. The second uses many piecewise continuous polynomials of low and fixed order, as in [15], these are h methods. The third combines both ideas, using many polynomials whose order is allowed to vary between segments, these are hp methods. The Legendre-Gauss, Legendre-Gauss-Radau and Legendre-Gauss-Lobatto schemes presented above are the building blocks for all three categories. A single global instance gives a p method. Chaining many low-order instances on consecutive intervals with continuity at the shared endpoints gives an h method. Chaining instances of different orders gives an hp method.

Intuitively p methods should suffer from the Runge phenomenon, but they do not. The discretization nodes are not equispaced, they are clustered toward the endpoints, which suppresses the oscillations. Methods based on roots of Chebyshev polynomials suppress the Runge phenomenon better than Legendre methods[68].

3.6 Mesh refinement methods

Mesh refinement is required to asses the accuracy of transcription, as optimization result without a guarantee on accuracy is useless. Mesh refinement works by changing polynomial order and number and place of segments. In mesh refinement it is also important to remove segments that are unnecessary. Mesh refinement methods are divided into three categories.

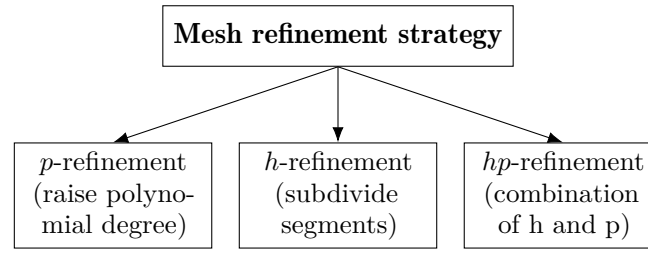


Figure 3.6: Mesh refinement categories [23, 58].

■ 3.6.1 P adaptive methods

The p adaptive methods work by increasing the order of polynomial, until the error is below user defined threshold.

■ 3.6.2 h adaptive methods

The h methods are used when solution of optimal control problem is approximated by many intervals with low degree polynomial. These methods can decide to split segment into two smaller segments, they use different techniques to do this. Method described in [15] calculates error using Romberg quadrature with machine precision. Another method described in [40] uses techniques from essentially non oscillatory (ENO) interpolation [34] to decide where to split and remove nodes.

■ 3.6.3 hp adaptive methods

The hp adaptive methods are state of the art in mesh refinement. They are combination of increasing/decreasing mesh segments and increasing/decreasing order of approximating polynomial inside segments. One of the first hp-method and commonly cited is described in article [23] from University of Florida. They have produced numerous papers on hp- adaptive mesh refinement [9] [50]. Recent work on mesh refinement for functions with kinks [75]. They are the most complex for implementation and unfortunately I did not have the time to implement them.

■ 3.7 Adjoint methods

Adjoint methods allow for differentiating the solution of optimization problem. In this case it is function of time on states and controls along the trajectory and car parameters. This is very useful as it greatly simplifies sensitivity analysis. Traditional way of sensitivity analysis is to compute several runs with different vehicle parameters, but this allows to compute sensitivity of all parameters in one shot. I am using DiffOpt.jl [14], it is not exactly adjoint I don't really understand very well how it works.

Chapter 4

Solvers for nonlinear programming

Once the objective function and constraints are created, the problem is passed to the NLP solver. Commonly used methods are Lagrange-Newton methods, these are sequential quadratic solvers and interior point solvers. These solvers are perhaps the most complex component used in this thesis, this chapter will cover them at high level, as full theory is beyond the scope. A comprehensive overview of Lagrange-Newton methods is in article [70], where the methods are broken down into key algorithmic components and unifying framework of solvers is presented. Figure 4.1 demonstrates the complexity of these methods.

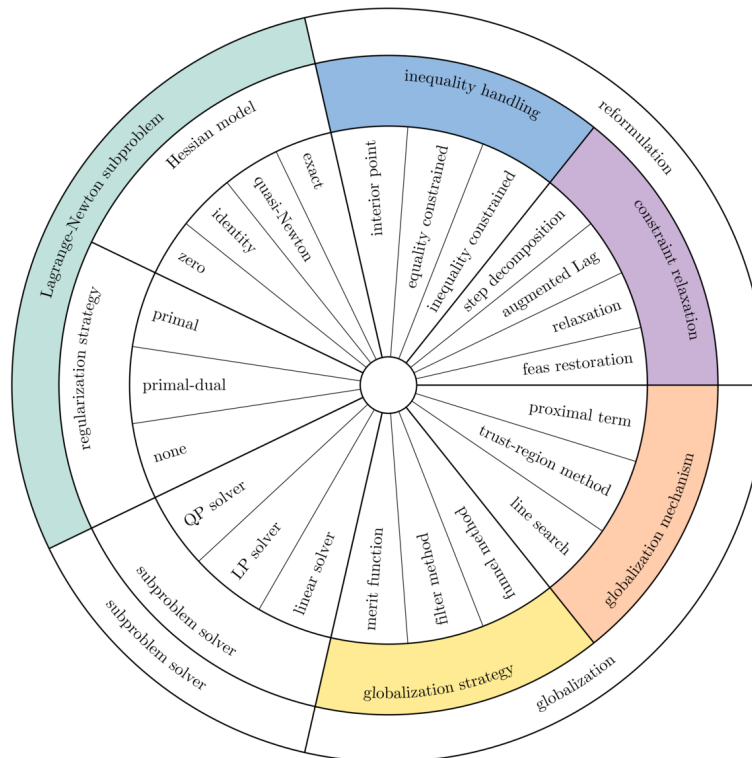


Figure 4.1: Unifying framework: the “wheel of strategies”[70]

High level overview required for this thesis of Lagrange-Newton methods is in Figure 4.2.

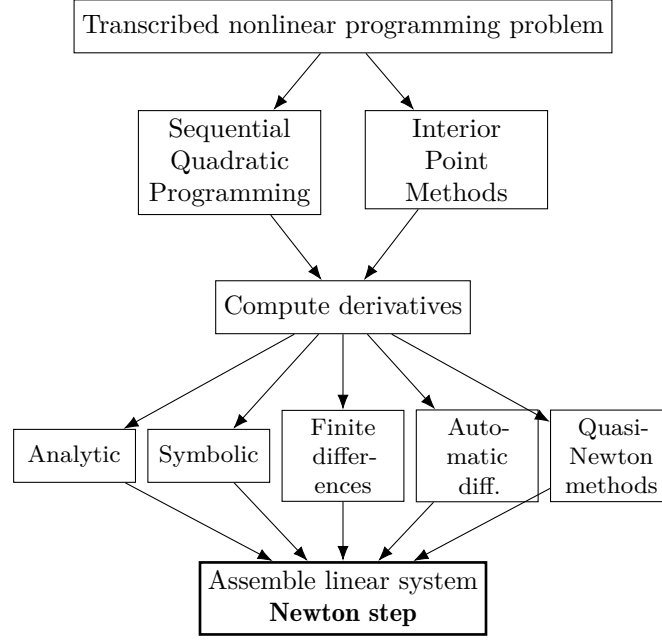


Figure 4.2: NLP solution flow

4.1 Nonlinear programming

General NLP problems can be written as:

$$\begin{aligned}
 \min_{\mathbf{x} \in \mathbb{R}^n} \quad & f(\mathbf{x}) \\
 \text{s.t.} \quad & \mathbf{h}(\mathbf{x}) = 0 \\
 & \mathbf{g}(\mathbf{x}) \leq 0
 \end{aligned} \tag{4.1}$$

Here $\mathbf{x} \in \mathbb{R}^n$ is the decision-variable vector, $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is the scalar objective, $\mathbf{h} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ stacks the m equality constraints, and $\mathbf{g} : \mathbb{R}^n \rightarrow \mathbb{R}^p$ stacks the p inequality constraints. In the lap-time setting $\mathbf{x}$ holds the discretised states and controls, f is total lap time, $\mathbf{h}$ enforces vehicle dynamics and boundary conditions, and $\mathbf{g}$ enforces track limits, tire friction circle and actuator bounds.

4.1.1 KKT conditions

KKT conditions are the conditions of optimality, if satisfied the point is local optimum. In nonlinear programming, we use the KKT conditions.

$$\begin{aligned}
 \nabla f(\mathbf{x}) + \sum_{i=1}^m \lambda_i \nabla h_i(\mathbf{x}) + \sum_{i=1}^p \mu_i \nabla g_i(\mathbf{x}) &= 0 \\
 \mathbf{h}(\mathbf{x}) &= \mathbf{0} \\
 \mathbf{g}(\mathbf{x}) &\leq \mathbf{0} \\
 \mu_i &\geq 0, \quad i = 1, \dots, p \\
 \mu_i g_i(\mathbf{x}) &= 0, \quad i = 1, \dots, p
 \end{aligned} \tag{4.2}$$

4.2 Interior point methods: IPOPT, MadNLP

Interior point solvers or barrier solvers work by reformulating the NLP into a barrier problem at each step and solving a sequence of these barrier problems, with decreasing barrier parameter. Solver used in this thesis is IPOPT [76] and MadNLP[64].

4.2.1 Barrier problem formulation

There are multiple ways to transform NLP in to barrier problem, this section will briefly describe how IPOPT does it. IPOPT article [76] Describes how to transform generic NLP into form 4.3

$$\begin{aligned}
 \min_{\mathbf{x} \in \mathbb{R}^n} \quad & \phi_\mu(\mathbf{x}) := f(\mathbf{x}) - \mu \sum_{i=1}^n \ln(x_i^{(i)}) \\
 \text{s.t.} \quad & c(\mathbf{x}) = 0
 \end{aligned} \tag{4.3}$$

The Lagrangian of the barrier problem is defined as

$$\mathcal{L}(\mathbf{x}, \lambda, z) := f(\mathbf{x}) + c(\mathbf{x})^\top \lambda - z. \tag{4.4}$$

Then barrier problem is further transformed into set of linear equations, a KKT linear system, that computes the Newton step:

$$\begin{bmatrix} W_k & A_k & -I \\ A_k^\top & 0 & 0 \\ Z_k & 0 & X_k \end{bmatrix} \begin{bmatrix} d_x^k \\ d_\lambda^k \\ d_z^k \end{bmatrix} = - \begin{bmatrix} \nabla f(x_k) + A_k \lambda_k - z_k \\ c(x_k) \\ X_k Z_k e - \mu_j e \end{bmatrix}. \tag{4.5}$$

Here $W_k = \nabla^2 \mathcal{L}(x_k, \lambda_k, z_k)$ is the Hessian of the Lagrangian, A_k is the Jacobian of the equality constraints $c(x)$ and $\nabla f(x_k)$ is the gradient of the objective. The unknowns d_x^k , d_λ^k and d_z^k are the Newton steps in the primal variables, equality multipliers and bound multipliers; μ_j is the barrier parameter, which is driven to zero across outer iterations.

There are numerous commercial and free algorithms to solve this system of linear equation. For smaller systems it is LU decomposition, but for larger sparse problems a dedicated solver is a must. IPOPT is most commonly used with MUMPS [10] solver which is free. There are several linear commercial solvers, Harwell Subroutine Library(HSL) offers ma27 ma87 ma97 [38].Panua

and linear solve may fail. This happens when for example one constraint is created twice.

4.6 Sparsity

Sparsity is a desirable property. Sparse Hessian and Jacobian matrices contain a large fraction of zero entries, which interior-point solvers exploit through specialized linear algebra. Sparsity in the constraints themselves is also important: individual constraints should involve only a small subset of the decision variables, rather than being formulated as deeply nested nonlinear expressions. Example of this is that state derivatives of vehicle at each point, can be fully determined from only 7 variables: accelerator pedal, brake pedal, steering angle, forward speed, lateral speed, yaw rate and heading. Using only these variables results in a deeply nested nonlinear expression in the constraints. Adding redundant decision variables like forces on all wheels can reduce the iterations needed and calculation time of derivatives. Counter intuitively this larger problem was faster to solve, this is elaborated on in [42, 43]. Certain solvers can heavily exploit the sparsity structure, such as FATROP[71]. In source [41] Joris Gillis a CasADi developer compares IPOPT to FATROP, with **FATROP being 33 times faster on a trajectory optimisation problem**. Also, [15] describes how sparsity structure helps with faster derivatives approximation with finite difference and Quasi-Newton methods.

4.7 Initialization

NLP solvers require initial guess for all decision variables, to start solving. For this problem initial guess is obtained driving the vehicle on given track. PD controller is used to keep the car on the centerline and P controller is used to keep velocity at 5m/s. In general, it is possible to initialize all variables at zero, but with large scale nonlinear programming this approach is not used. For this problem initialization with zeros leads to singularity in equation 2.38 and is not possible at all.

4.8 Global and local optimums

This optimization problem is generally non-convex, therefore any local optimum does not guarantee global optimum. Commonly used NLP solvers, like IPOPT guarantee only local optimums and not global ones. Good initialization is basis for finding a local optimum close to the global optimum [48, §5.1, p. 11].



Part II

Implementation

Chapter 5

Frameworks for Optimal Control Problem Formulation

Process of solving optimal control problem has 3 parts. Problem definition, solving nonlinear program and optional mesh refinement. This chapter describes frameworks in which the NLP problem can be defined and solved. Framework is: programming language + algebraic modeling tool + automatic differentiation tool + mesh refining algorithm. Solving optimal control problem can be split into 3 steps described in figure 5.1

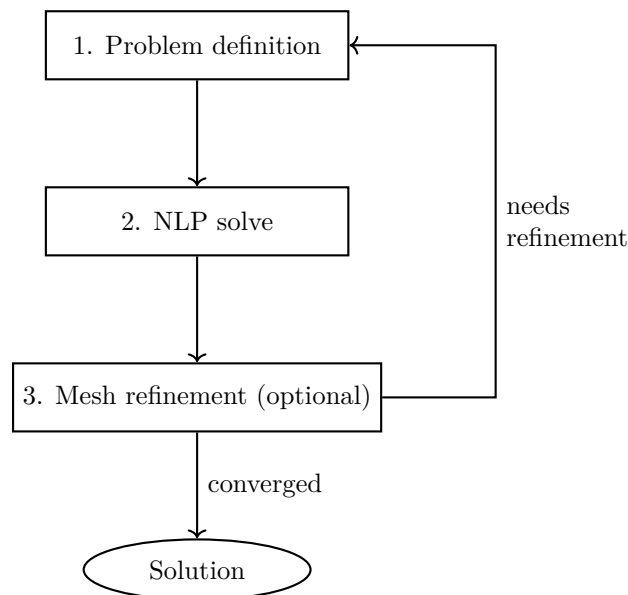


Figure 5.1: Optimal control problem solving pipeline. Adapted from [15, p. 91]

The consensus in optimal control community about tools for problem formulation is that:

There is no shortage of optimal control software based on pseudospectral approaches. There are a number of other optimal control libraries that tackle similar kinds of problems, such as OTIS4, GPOPS-II, and CASADI. [27, 26]

CasADi is an open-source tool for nonlinear optimization and algorithmic differentiation.[17][11]. interfaces with C++,Python,MATLAB. It provides a framework to build the problem, and calculate the hessian and Jacobian for NLP solver it does not have any mesh refinement algorithm. Commonly paired with Ipopt, but is compatible with large variety of solvers.

OpenMDAO [32] was developed at NASA Glenn research center in python for evaluating aircraft concepts. Dymos implements hp mesh refinement and Lobatto and radau collocation as default [27]. According to forum post from julia discourse, which may be biased towards julia it is slower compared to infiteOpt in julia [6]. It is the only open source tool I found that offers mesh refinement.

Acados is a modular open-source framework for fast embedded optimal control, developed at University of Freiburg [72]. It has C, Python, MATLAB + Simulink and Octave interface. This framework supports generating C code. It is demonstrated by numerous real-world examples, such as drone racing with *replanning*, robot navigation in cluttered environment and many others [8].

GPOPS-II [59] is perhaps the most advanced tool for optimal control, it was developed at University of Florida by Michael Patterson and Anil V. Rao. other tools allows defining the problem, but GPOPS-II is a level higher. It also includes advanced mesh refinement strategies, multiple phase OCP and automatic scaling. For solving the problem it uses Ipopt or SNOPT. It interfaces with MATLAB and C++, uses ADigator [73] for automatic differentiation. This tool is commercial and closed source. Implementation in this thesis is heavily based on work of community from University of Florida.

ExaModels, JuMP

JuMP is the go to tool for algebraic modeling in julia, it is mature, versatile. Supports Linear programming, quadratic programming, mixed linear programming, Nonlinear programming. ExaModels [66] is a specialized algebraic

modeling tool, it utilizes Single instruction, multiple data methods, enabling derivative evaluation on GPU accelerators. Coupled with MadNLP solver, the entire computation of nonlinear optimization can be run on a GPU. No julia package has mesh refinement. Julia offers several framework options

1. Julia + jump
2. Julia + jump + infinte opt
3. julia + ExaModels

Although package InfiteOpt.jl already contains implementation of various collocation methods, I chose not to use it. This decision was motivated by two considerations. Implementing the transcription methods myself provides deeper understanding of the topic. InfiteOpt abstracts away some of the low lever control that JuMP offers, which may prevent mesh refinement implementation, usage of parametric vehicle model or sensitivity analysis.

I chose julia + jump,



Chapter 6

My implementation, Julia + JuMP

I chose Julia with JuMP as the modeling environment. The toolchain had to be free and open source, fast and ergonomic. Julia targets the gap between C++ performance and Python ergonomics, and its community packages provide ready-to-use numerical tools. I also had prior experience in Python and wanted to learn Julia. My implementation is available on github <https://github.com/ceserik/SLapSim.jl>.

JuMP [51] lets the user declare decision variables and constraints symbolically and uses automatic differentiation to supply first- and second-order derivatives to the solver. Through JuMP I use two NLP solvers: IPOPT [76] for the CPU path and MadNLP [65, 64] for both CPU and GPU paths. The full optimisation stack is shown in Figure 6.1.

The package InfiniteOpt.jl already implements several collocation schemes, but I chose to implement transcription myself. First, doing so gave a deeper understanding of the underlying methods. Second, InfiniteOpt abstracts away the low-level JuMP access, which may make custom mesh refinement, parametric vehicle models, and sensitivity analysis more difficult.

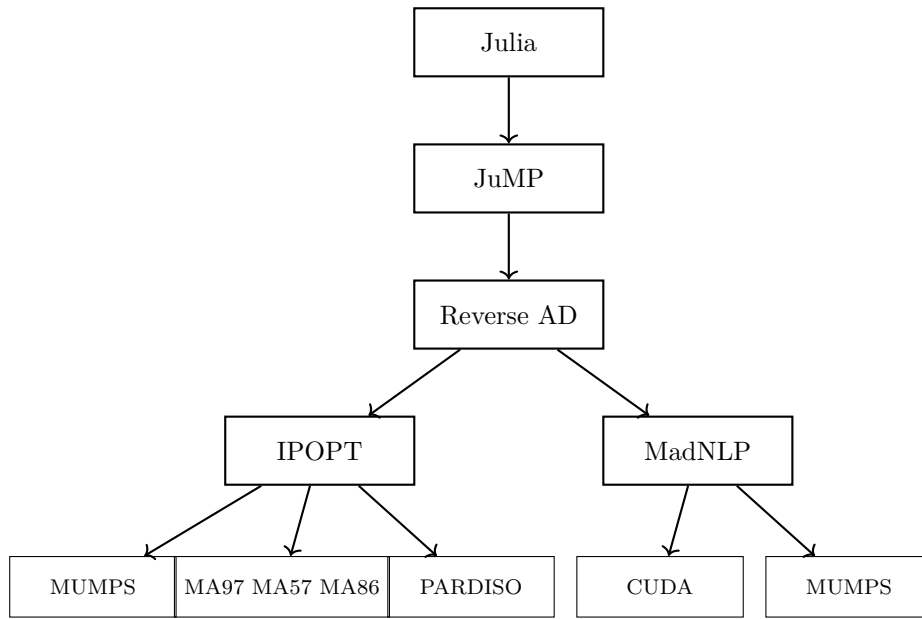


Figure 6.1: Optimization software stack

6.1 sensitivity Analysis

I have used DiffOpt.jl [14] for the sensitivity analysis. By adding vehicle parameters such as, mass, lift coefficient, CoG position etc. into optimization as decisions variables fixed to a value by constraints. This added them to the algebraic model. Then the solution of minimum laptime can be differentiated by these parameters. Result of sensitivity analysis are on figure 6.2.

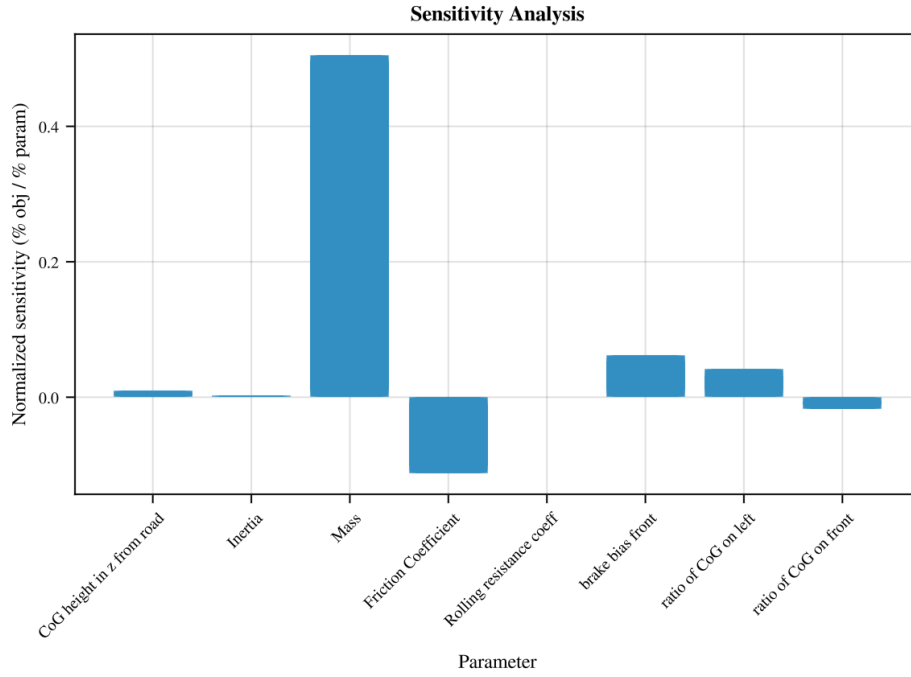


Figure 6.2: Node-interval ODE error for power-test case.

6.2 Implemented nonlinear programming solvers

Julia with JuMP offer high level interface which allows switching NLP solvers on single line. I have used Ipopt, MadNLP and Uno solver with preset filtersqp. Unfortunately the Uno solver was getting stuck, therefore it is not considered in this thesis.

Ipopt has built in Hessian approximation strategy, in benchmark it proved to be inferior to automatic differentiation. Figure 6.3 shows which solving methods were implemented and tested.

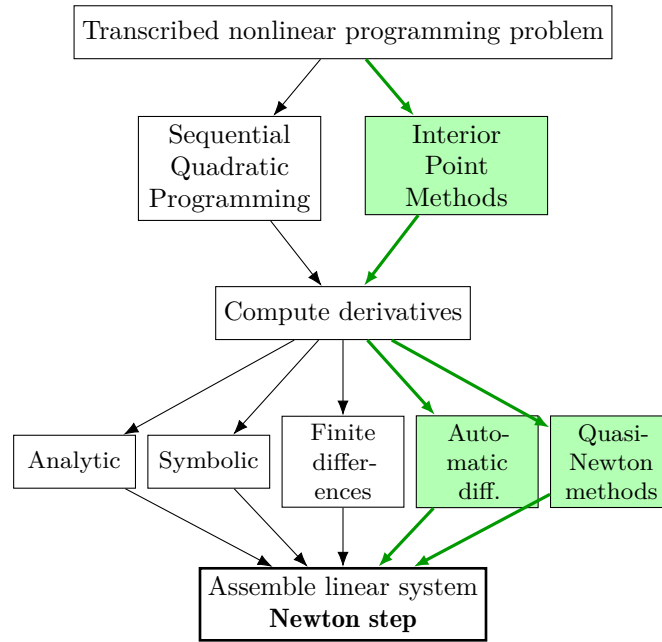


Figure 6.3: NLP solution flow with the methods used in this thesis highlighted in green

6.3 Implementation of mesh refinement

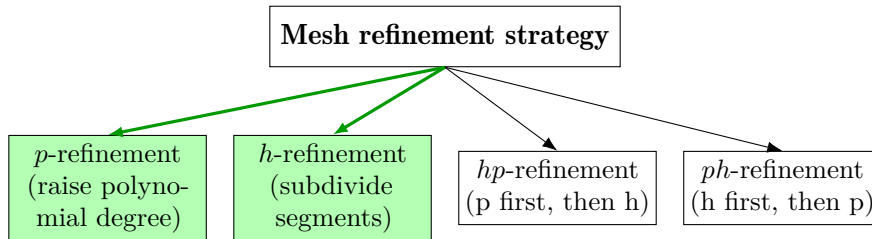


Figure 6.4: Mesh refinement layer with the strategies implemented in this thesis highlighted in green

I have implemented p and h method for mesh refinement. Error calculation is implemented using comparing the result of interpolated value at the end of segment and the result of simulation using Rodas4() solver. To calculate the error at the end of each segment, the interpolated controls and initial states are plugged into Rodas4 IVP solver. Then this result is compared to the states from transcription method. If error between states in absolute value is larger than user defined threshold, the segment is halved. This is one of the simpler estimators of discretization error.

Figure 6.5 shows that error along track is not distributed equally. Error is localized around areas with large jump in controls. These areas need finer sampling. P methods cannot focus these specific points directly, they focus entire trajectory.

The errors are concentrated around the discontinuity of controls, where the behaviour of the vehicle is more complex/ requiring high order or finer approximation see Figure 6.5. Time optimal control of car around track has many discontinuities and therefore p methods are not a good fit for this use.

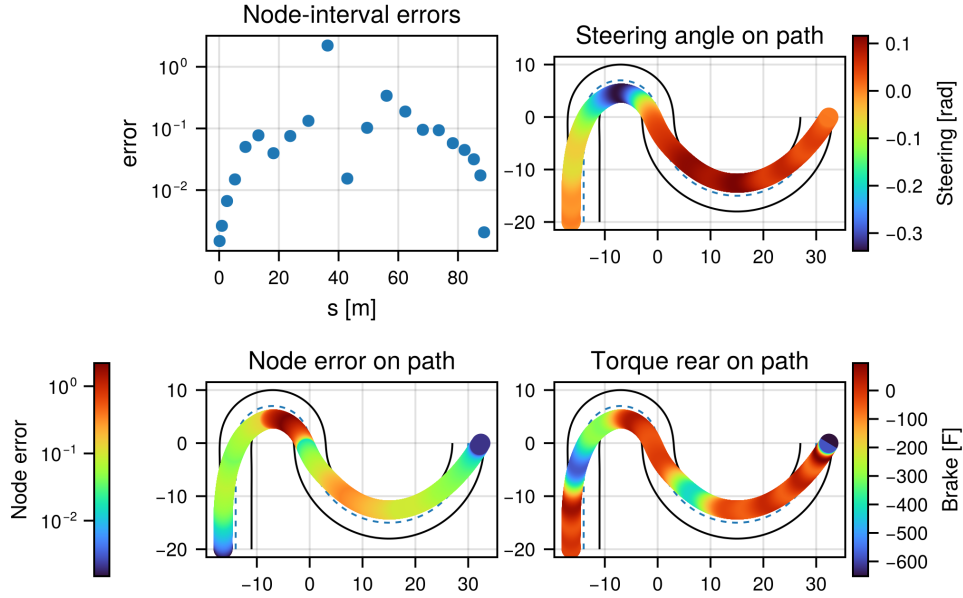


Figure 6.5: Node-interval ODE error for power-test case.

The h methods handle this better by increasing sampling density around segments with large errors. Error before refinement is on figure 6.6.

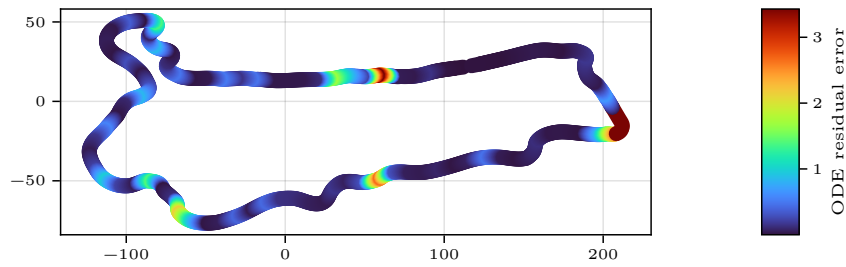


Figure 6.6: Discretization error before mesh refinement

Mesh density after refinement is on figure 6.7

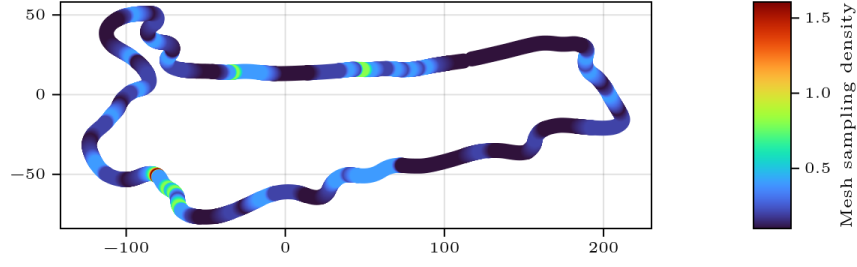


Figure 6.7: Sampling density after mesh refinement

6.4 Implementation of collocation methods

I have implemented Legendre-Gauss(LG), Legendre-Gauss-Radau(LGR) and Legendre-Gauss-Lobatto(LGL) methods. LG and LGR are implemented in differential formulation and LGL in integral formulation. Implementation supports an arbitrary number of collocation nodes. The integral formulation is chosen for LGL because, with two and three nodes, it is reduced to the trapezoidal and Hermite-Simpson methods. A significant speed-up of evaluation of automatic differentiation was obtained by creating more decision variables, thus the nonlinear constraints were less dense [42].

These methods are quite similar in their implementation in julia, I have attempted to create a unified implementation using a single function, but I was unsuccessful. In the future, having such implementation would be beneficial to test a wider variety of collocation methods. Figure 6.8 shows methods that were implemented.

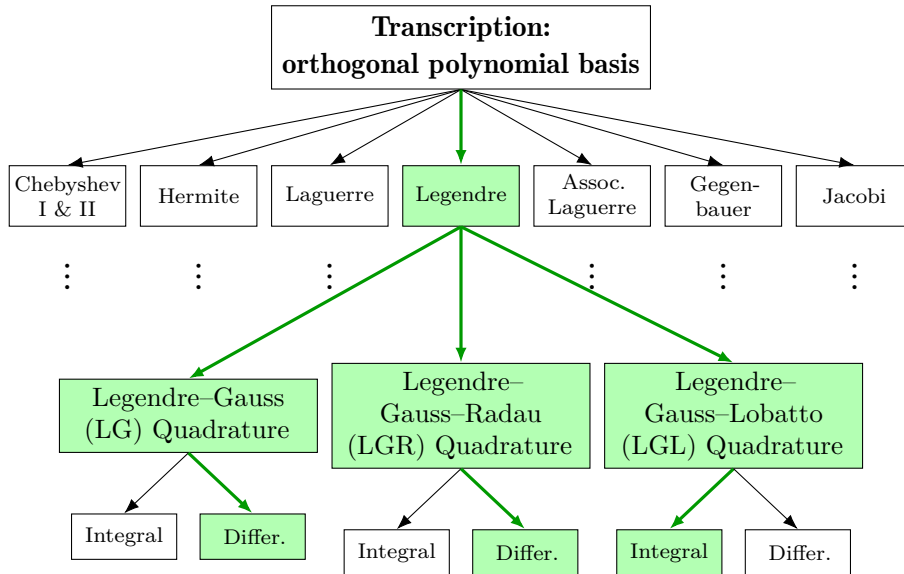


Figure 6.8: Transcription layer with the methods used in this thesis highlighted in green

6.5 Implementation of models

6.5.1 Tire model implementation

Three tire models were implemented according to section 2.2. Extra decision variables were created that are equal to tire lateral and longitudinal forces, this creates less dense nonlinear expressions. Tires have these parametrizations:

Table 6.1: Tire model parameters.

Linear R20		Pacejka R20	
Parameter	Value	Parameter	Value
Radius R [m]	0.205	Radius R [m]	0.205
Width [m]	0.30	Width [m]	0.30
Max slip angle $\alpha_{\max}$ [deg]	5	Friction coefficient μ [-]	1.0
Friction coefficient μ [-]	1.0	Rolling resistance c_r [-]	0.010
Rolling resistance c_r [-]	0.010	Stiffness factor B	9.62
		Shape factor C	2.59
		Curvature factor E	1.0
		Peak factor D	μF_z

Pacejka Formula E	
Parameter	Value
Radius R [m]	0.33
Width [m]	0.30
Friction coefficient μ [-]	1.0
Rolling resistance c_r [-]	0.010
Stiffness factor B	9.62
Shape factor C	2.59
Curvature factor E	1.0
Peak factor D	$\mu F_z \left(1 - \frac{0.0813 F_z}{3000}\right)$

6.5.2 Implementation of vehicles

Four vehicles were implemented. The simplest is the single-track Formula Student vehicle, with a static suspension 2.6.1 and a linear tire 2.2.1. The controls are the braking force, the rear motor torque, and the steering of the front wheels.

The twin-track Formula Student vehicle has a simplified Magic Formula tire model 2.2.2 without the degressive behavior and a static suspension 2.6.1. The controls are the torque on the front motors, the torque on the rear motors, the braking force, and the steering of the front wheels. It is implemented according to [1].

The twin-track Formula E model has a Magic Formula tire model 2.2.2 with degressive behavior and a dynamic suspension. The controls are the torque on the rear wheels, the braking force, and the steering of the front wheels.

The bus model has a scaled-up Magic Formula tire model and a dummy suspension that splits the normal load equally. The controls are the torque

on the middle axle, the independent steering of the front and rear wheels, and the braking force.

Table 6.2: Vehicle parameters used in this thesis

Parameter	Singletrack	FS Twintrack	Formula E	Bus
Mass m [kg]	280	280	1200	12000
Inertia I_z [kg m ²]	100	100	1260	30000
Wheelbase L [m]	1.525	1.525	2.9	6.0
Track w [m]	1.2	1.2	1.5	2.1
CoG ratio front [-]	0.50	0.50	0.482	0.45
Lift coefficient C_L [-]	5.0	5.0	-2.7	-0.2
Drag coefficient C_D [-]	2.0	2.0	1.4	6.0
Power limit $P_{\max}$ [kW]	80	80	270	300
Brake bias front [-]	0.60	0.60	0.70	0.60
Max brake force [kN]	20	20	20	500
Velocity range [m/s]	[3,60]	[3,60]	[2,60]	[2,30]
Wheels	2	4	4	6
Driven motors	2	4	1 (shared)	6
Tire model	linear R20	Pacejka R20	Pacejka FE	Pacejka (scaled)
Suspension	Static	Static	Dynamic	dummy 6-wheel

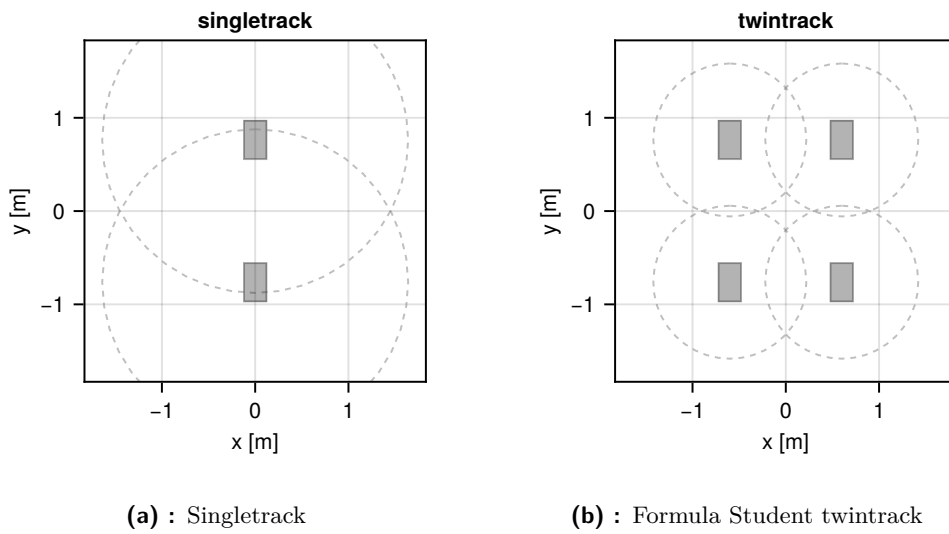


Figure 6.9: Single-track and Formula Student twintrack vehicle models.

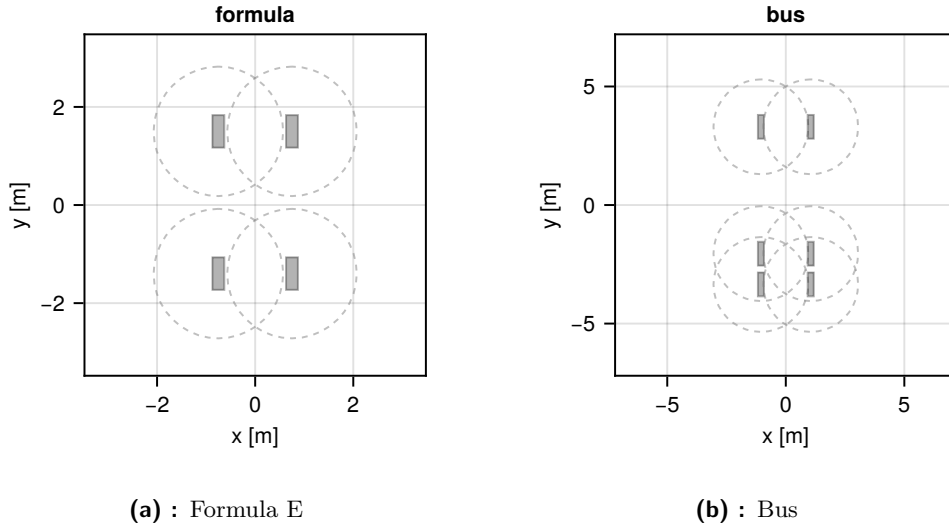


Figure 6.10: Formula E and Bus vehicle models.

6.6 Benchmark tracks

The following tracks are used in the benchmark in Section 7. They cover real Formula Student layouts (FSCZ, FSG), a Formula E street circuit (Berlin) and two synthetic geometries (DoubleTurn, FigureEight).

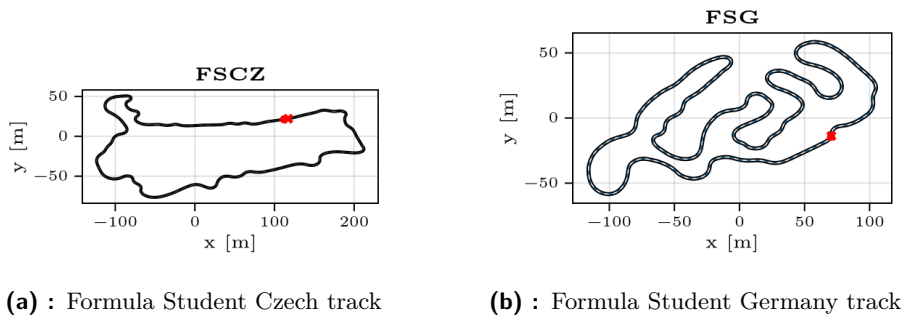


Figure 6.11: Formula Student benchmark tracks.

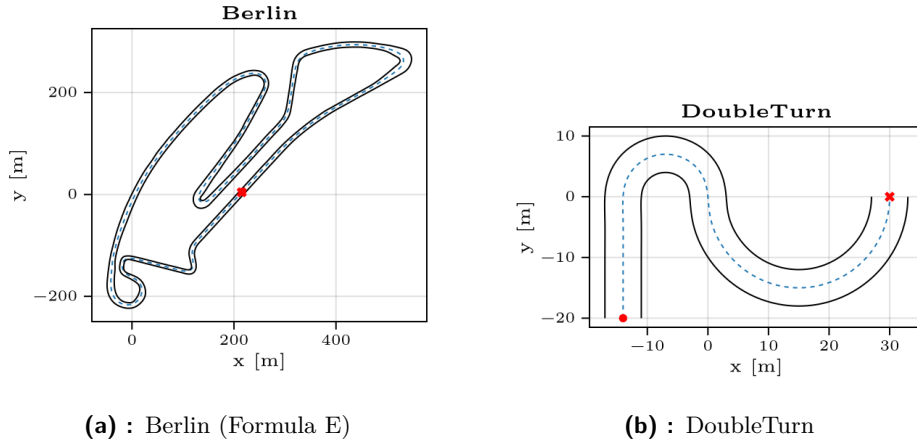


Figure 6.12: Berlin Formula E circuit and the DoubleTurn synthetic track.

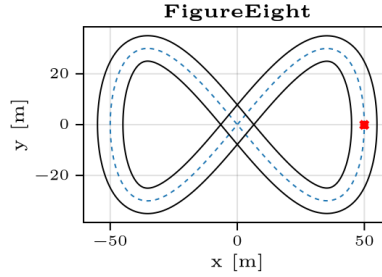


Figure 6.13: FigureEight synthetic track.

6.7 Example optimization problem

After transcription the OCP becomes a nonlinear program over the state and control samples at the collocation nodes,

$$\begin{aligned}
 & \min_{\mathbf{X}, \mathbf{U}} t_f \\
 & \text{s.t.} \quad \mathbf{DX} = \mathbf{F}(\mathbf{X}, \mathbf{U}), && \text{collocated dynamics} \\
 & \quad \mathbf{G}(\mathbf{X}, \mathbf{U}) \leq 0, && \text{path constraints at nodes} \\
 & \quad \mathbf{X}_0 = \mathbf{x}_0, \quad \mathbf{X}_N = \mathbf{x}_f, && \text{boundary conditions} \\
 & \quad \mathbf{X}_{\text{lb}} \leq \mathbf{X} \leq \mathbf{X}_{\text{ub}}, && \text{state bounds} \\
 & \quad \mathbf{U}_{\text{lb}} \leq \mathbf{U} \leq \mathbf{U}_{\text{ub}}. && \text{control bounds}
 \end{aligned} \tag{6.1}$$

the objective is to minimize the final time.



Part III

Results

Chapter 7

Comparison of methods

The performance of NLP solvers and frameworks is often tested on benchmark problems. like CUTEst [31] dataset. I have created my own dataset for this purpose. As written on Ipopt website: Ipopt has many (probably too many) options [21] that can be adjusted, and that is true for all solvers. Some of differences in this benchmark may be due to sub optimal setup of the solver. Mesh refinement method in these tests is the h method with error threshold 10^{-1} Tests were performed on Ryzen 9 9950X and MadNLP ran on Nvidia RTX 5070. This benchmark tests the time to solve and solve failure rate for all implemented vehicle variants on benchmarking tracks. Videos of some of the benchmarks are available on the YouTube playlist [20].

7.1 Benchmark of collocation methods

This benchmark tests Collocation methods. Legendre and Lobatto methods failed in one of 14 tests Radau did not fail, but took little longer than Lobatto method. Interestingly the lap-time seems to vary in some cases by around 0.3 second on same track and same vehicle with different transcription method.

transcription	runs	ok	fail	rate	mean_s	med_s	min_s	max_s
Legendre	14	13	1	92.8571	115.5586	25.9074	0.5774	1009.4537
Lobatto IIIA	14	13	1	92.8571	66.4536	22.3981	0.2790	532.3975
Radau	14	14	0	100.0000	71.6885	21.5821	0.4218	572.8931

Table 7.2: Benchmark summary statistics by transcription variant.

track	car	transcription	ok	status	time_s	obj
FSCZ	singletrack	Radau	yes	solved	29.1453	50.2106
FSCZ	singletrack	Legendre	yes	solved	49.7079	50.5890
FSCZ	singletrack	Lobatto IIIA	yes	solved	28.0042	51.4710
FSCZ	twintrack	Radau	yes	solved	109.6182	57.3381
FSCZ	twintrack	Legendre	no	locally infeasible	381.2216	NaN
FSCZ	twintrack	Lobatto IIIA	no	locally infeasible	224.8871	NaN
Berlin	singletrack	Radau	yes	solved	25.9621	79.4234
Berlin	singletrack	Legendre	yes	solved	44.6565	79.4033
Berlin	singletrack	Lobatto IIIA	yes	solved	39.2681	79.1299
Berlin	twintrack	Radau	yes	solved	67.7176	77.7439
Berlin	twintrack	Legendre	yes	solved	134.5996	77.7883

Berlin	twintrack	Lobatto IIIA	yes	solved	52.1696	77.8983
Berlin	formula	Radau	yes	solved	65.8456	87.9477
Berlin	formula	Legendre	yes	solved	96.1020	88.3235
Berlin	formula	Lobatto IIIA	yes	solved	76.7914	87.9482
Berlin	bus	Radau	yes	solved	572.8931	106.9827
Berlin	bus	Legendre	yes	solved	1009.4537	106.6620
Berlin	bus	Lobatto IIIA	yes	solved	532.3975	105.8631
FSG	singletrack	Radau	yes	solved	17.2021	58.2463
FSG	singletrack	Legendre	yes	solved	25.9074	58.1609
FSG	singletrack	Lobatto IIIA	yes	solved	22.3981	56.8773
FSG	twintrack	Radau	yes	solved	86.4238	69.3149
FSG	twintrack	Legendre	yes	solved	109.9323	70.3348
FSG	twintrack	Lobatto IIIA	yes	solved	95.6528	69.8878
DoubleTurn	singletrack	Radau	yes	solved	0.4218	5.7087
DoubleTurn	singletrack	Legendre	yes	solved	0.5774	5.6794
DoubleTurn	singletrack	Lobatto IIIA	yes	solved	0.2790	6.0247
DoubleTurn	twintrack	Radau	yes	solved	2.1042	6.5283
DoubleTurn	twintrack	Legendre	yes	solved	1.2756	6.5835
DoubleTurn	twintrack	Lobatto IIIA	yes	solved	1.5196	7.0205
DoubleTurn	bus	Radau	yes	solved	2.2998	8.9075
DoubleTurn	bus	Legendre	yes	solved	1.9180	8.9864
DoubleTurn	bus	Lobatto IIIA	yes	solved	1.8477	8.8676
FigureEight	singletrack	Radau	yes	solved	1.5903	14.9127
FigureEight	singletrack	Legendre	yes	solved	1.9768	14.9162
FigureEight	singletrack	Lobatto IIIA	yes	solved	1.3477	14.8296
FigureEight	twintrack	Radau	yes	solved	12.5541	16.5690
FigureEight	twintrack	Legendre	yes	solved	20.7391	16.5736
FigureEight	twintrack	Lobatto IIIA	yes	solved	9.2105	16.5319
FigureEight	bus	Radau	yes	solved	9.8610	22.8420
FigureEight	bus	Legendre	yes	solved	5.4157	22.7966
FigureEight	bus	Lobatto IIIA	yes	solved	3.0111	22.3442

Table 7.4: Benchmark results by track, vehicle model, and transcription variant.

7.2 Benchmark of Linear solvers

Surprisingly free MUMPS solver performed better than solvers from proprietary Harwell Subroutine Library. In this test I did not include PARDISO as my license was limited only to two cores.

linear solver	runs	ok	fail	rate	mean_s	med_s	min_s	max_s
ma27	14	14	0	100.0000	97.1562	12.1841	0.2777	964.2969
ma57	14	14	0	100.0000	93.3249	12.6211	0.2897	895.8892
ma97	14	14	0	100.0000	107.8802	12.1752	0.2689	1099.4816
mumps	14	14	0	100.0000	69.6295	12.6470	0.2803	569.8588

Table 7.6: Linear solver benchmark summary statistics.

track	car	linear solver	ok	status	time_s	obj
FSCZ	singletrack	mumps	yes	solved	11.4861	50.2106
FSCZ	singletrack	ma57	yes	solved	11.2823	50.2106
FSCZ	singletrack	ma27	yes	solved	11.2823	50.2106
FSCZ	singletrack	ma97	yes	solved	11.2701	50.2106

FSCZ	twintrack	mumps	yes	solved	108.9638	57.3381
FSCZ	twintrack	ma57	yes	solved	91.0531	57.3381
FSCZ	twintrack	ma27	yes	solved	108.9439	57.2790
FSCZ	twintrack	ma97	yes	solved	92.4024	57.3381
Berlin	singletrack	mumps	yes	solved	26.6890	79.4234
Berlin	singletrack	ma57	yes	solved	24.6851	79.4234
Berlin	singletrack	ma27	yes	solved	25.5493	79.4234
Berlin	singletrack	ma97	yes	solved	25.6775	79.4234
Berlin	twintrack	mumps	yes	solved	67.2587	77.7439
Berlin	twintrack	ma57	yes	solved	58.9130	77.7439
Berlin	twintrack	ma27	yes	solved	51.7979	77.7439
Berlin	twintrack	ma97	yes	solved	55.8040	77.7439
Berlin	formula	mumps	yes	solved	64.5359	87.9477
Berlin	formula	ma57	yes	solved	62.7170	87.9477
Berlin	formula	ma27	yes	solved	60.3951	87.9477
Berlin	formula	ma97	yes	solved	73.0484	87.9477
Berlin	bus	mumps	yes	solved	569.8588	106.9827
Berlin	bus	ma57	yes	solved	895.8892	106.9795
Berlin	bus	ma27	yes	solved	964.2969	106.9815
Berlin	bus	ma97	yes	solved	1099.4816	106.6410
FSG	singletrack	mumps	yes	solved	12.7246	58.2463
FSG	singletrack	ma57	yes	solved	13.9599	58.2463
FSG	singletrack	ma27	yes	solved	13.0858	58.2463
FSG	singletrack	ma97	yes	solved	13.0492	58.2463
FSG	twintrack	mumps	yes	solved	85.2451	69.3149
FSG	twintrack	ma57	yes	solved	123.3800	69.3471
FSG	twintrack	ma27	yes	solved	100.8860	69.3140
FSG	twintrack	ma97	yes	solved	115.3881	69.3140
DoubleTurn	singletrack	mumps	yes	solved	0.2803	5.7087
DoubleTurn	singletrack	ma57	yes	solved	0.2897	5.7087
DoubleTurn	singletrack	ma27	yes	solved	0.2777	5.7087
DoubleTurn	singletrack	ma97	yes	solved	0.2689	5.7087
DoubleTurn	twintrack	mumps	yes	solved	2.0312	6.5283
DoubleTurn	twintrack	ma57	yes	solved	2.8587	6.5283
DoubleTurn	twintrack	ma27	yes	solved	1.8017	6.5283
DoubleTurn	twintrack	ma97	yes	solved	1.9207	6.5283
DoubleTurn	bus	mumps	yes	solved	2.2462	8.9075
DoubleTurn	bus	ma57	yes	solved	1.5365	8.9075
DoubleTurn	bus	ma27	yes	solved	1.3301	8.9075
DoubleTurn	bus	ma97	yes	solved	1.3733	8.9075
FigureEight	singletrack	mumps	yes	solved	0.8838	14.9127
FigureEight	singletrack	ma57	yes	solved	0.8841	14.9127
FigureEight	singletrack	ma27	yes	solved	0.9921	14.9127
FigureEight	singletrack	ma97	yes	solved	0.9997	14.9127
FigureEight	twintrack	mumps	yes	solved	12.5693	16.5690
FigureEight	twintrack	ma57	yes	solved	11.0038	16.5690
FigureEight	twintrack	ma27	yes	solved	11.1527	16.5690
FigureEight	twintrack	ma97	yes	solved	11.3013	16.5690
FigureEight	bus	mumps	yes	solved	10.0401	22.8420
FigureEight	bus	ma57	yes	solved	8.0956	22.8420
FigureEight	bus	ma27	yes	solved	8.3957	22.8420
FigureEight	bus	ma97	yes	solved	8.3372	22.8420

Table 7.8: Linear solver benchmark results by track, vehicle model, and transcription variant.

7.3 Benchmark of NLP solvers

I have tested Ipopt with MUMPS and madnlp with MUMPS and with CuDSS solver. These solvers were suspiciously similar in computation time on CPU and GPU was slower. with madnlp-gpu variant the automatic differentiation was calculated on CPU using JuMP and the solver ran on GPU [67], so it was not a 100% GPU solution.

NLP solver	runs	ok	fail	rate	mean_s	med_s	min_s	max_s
ipopt-mumps	14	14	0	100.0000	126.2746	12.3629	0.2810	1347.3147
madnlp-cpu	14	14	0	100.0000	126.0898	17.1029	0.2816	1370.6252
madnlp-gpu	14	14	0	100.0000	155.0950	20.4672	0.4655	1733.7308

Table 7.10: NLP solver / backend benchmark summary statistics (IPOPT-MUMPS, MadNLP-CPU, MadNLP-GPU).

track	car	NLP solver	ok	status	time_s	obj
FSCZ	singletrack	ipopt-mumps	yes	solved	11.5065	50.2106
FSCZ	singletrack	madnlp-cpu	yes	solved	18.3244	50.2106
FSCZ	singletrack	madnlp-gpu	yes	solved	70.2315	50.2098
FSCZ	twintrack	ipopt-mumps	yes	solved	109.2145	57.3381
FSCZ	twintrack	madnlp-cpu	yes	solved	103.5061	57.2490
FSCZ	twintrack	madnlp-gpu	yes	solved	92.1590	57.2521
Berlin	singletrack	ipopt-mumps	yes	solved	25.6021	79.4234
Berlin	singletrack	madnlp-cpu	yes	solved	18.5674	79.4234
Berlin	singletrack	madnlp-gpu	yes	solved	26.4438	79.4214
Berlin	twintrack	ipopt-mumps	yes	solved	87.9163	77.7417
Berlin	twintrack	madnlp-cpu	yes	solved	59.0629	77.7442
Berlin	twintrack	madnlp-gpu	yes	solved	45.7640	77.7421
Berlin	formula	ipopt-mumps	yes	solved	64.0543	87.9477
Berlin	formula	madnlp-cpu	yes	solved	60.2524	87.9477
Berlin	formula	madnlp-gpu	yes	almost locally solved	77.4451	87.9191
Berlin	bus	ipopt-mumps	yes	solved	1347.3147	107.2120
Berlin	bus	madnlp-cpu	yes	solved	1370.6252	106.6728
Berlin	bus	madnlp-gpu	yes	solved	1733.7308	106.8060
FSG	singletrack	ipopt-mumps	yes	solved	12.3721	58.2463
FSG	singletrack	madnlp-cpu	yes	solved	13.4788	58.2463
FSG	singletrack	madnlp-gpu	yes	solved	13.4315	58.2455
FSG	twintrack	ipopt-mumps	yes	solved	83.2545	69.3149
FSG	twintrack	madnlp-cpu	yes	solved	90.3133	69.4039
FSG	twintrack	madnlp-gpu	yes	solved	85.6687	69.3864
DoubleTurn	singletrack	ipopt-mumps	yes	solved	0.2810	5.7087
DoubleTurn	singletrack	madnlp-cpu	yes	solved	0.2816	5.7087
DoubleTurn	singletrack	madnlp-gpu	yes	solved	0.4655	5.7086
DoubleTurn	twintrack	ipopt-mumps	yes	solved	1.9413	6.5283
DoubleTurn	twintrack	madnlp-cpu	yes	solved	2.6051	6.5463
DoubleTurn	twintrack	madnlp-gpu	yes	solved	2.3967	6.5282
DoubleTurn	bus	ipopt-mumps	yes	solved	1.4252	8.9075
DoubleTurn	bus	madnlp-cpu	yes	solved	1.3106	8.9075
DoubleTurn	bus	madnlp-gpu	yes	solved	1.9968	8.8951
FigureEight	singletrack	ipopt-mumps	yes	solved	0.8828	14.9127
FigureEight	singletrack	madnlp-cpu	yes	solved	0.8406	14.9127
FigureEight	singletrack	madnlp-gpu	yes	solved	1.2380	14.9124
FigureEight	twintrack	ipopt-mumps	yes	solved	12.3536	16.5690
FigureEight	twintrack	madnlp-cpu	yes	solved	15.8815	16.5658

FigureEight	twintrack	madnlp-gpu	yes	solved	5.8682	16.5652
FigureEight	bus	ipopt-mumps	yes	solved	9.7260	22.8420
FigureEight	bus	madnlp-cpu	yes	solved	10.2077	22.7936
FigureEight	bus	madnlp-gpu	yes	solved	14.4905	22.7981

Table 7.12: NLP solver / backend benchmark results by track, vehicle model, and transcription variant.

7.4 Benchmark of derivatives computation

This section compares two ways to provide the Hessian to Ipopt. Exact second-order derivatives obtained through JuMP's automatic differentiation are compared against Ipopt's built-in limited-memory quasi-Newton approximation. Exact Hessian was faster in all tested cases. The quasi-Newton approximation also failed to converge in time on some of the harder problems.

hessian mode	runs	ok	fail	rate	mean_s	med_s	min_s	max_s
ipopt-exact	14	14	0	100.0000	124.7120	13.5311	0.2756	1317.5593
ipopt-lbfgs	14	7	7	50.0000	12.8208	14.0032	0.2919	29.6997

Table 7.14: Hessian approximation benchmark summary statistics (exact vs. limited-memory quasi-Newton).

track	car	hessian mode	ok	status	time_s	obj
FSCZ	singletrack	ipopt-exact	yes	solved	11.5460	50.2106
FSCZ	singletrack	ipopt-lbfgs	yes	solved	14.0032	50.2106
FSCZ	twintrack	ipopt-exact	yes	solved	107.1464	57.3381
FSCZ	twintrack	ipopt-lbfgs	no	iteration limit	797.5930	NaN
Berlin	singletrack	ipopt-exact	yes	solved	27.4134	79.4234
Berlin	singletrack	ipopt-lbfgs	yes	solved	29.6997	79.4234
Berlin	twintrack	ipopt-exact	yes	solved	89.4514	77.7417
Berlin	twintrack	ipopt-lbfgs	no	iteration limit	2547.2935	NaN
Berlin	formula	ipopt-exact	yes	solved	66.6850	87.9477
Berlin	formula	ipopt-lbfgs	no	iteration limit	796.0425	NaN
Berlin	bus	ipopt-exact	yes	solved	1317.5593	107.2120
Berlin	bus	ipopt-lbfgs	no	iteration limit	695.7941	NaN
FSG	singletrack	ipopt-exact	yes	solved	14.5996	58.2463
FSG	singletrack	ipopt-lbfgs	yes	solved	15.0913	58.2463
FSG	twintrack	ipopt-exact	yes	solved	84.5573	69.3149
FSG	twintrack	ipopt-lbfgs	no	iteration limit	1387.7969	NaN
DoubleTurn	singletrack	ipopt-exact	yes	solved	0.2756	5.7087
DoubleTurn	singletrack	ipopt-lbfgs	yes	solved	0.2919	5.7087
DoubleTurn	twintrack	ipopt-exact	yes	solved	1.9790	6.5283
DoubleTurn	twintrack	ipopt-lbfgs	yes	almost locally solved	21.8729	6.5412
DoubleTurn	bus	ipopt-exact	yes	solved	1.4282	8.9075
DoubleTurn	bus	ipopt-lbfgs	yes	solved	7.7314	8.9075
FigureEight	singletrack	ipopt-exact	yes	solved	0.8776	14.9127
FigureEight	singletrack	ipopt-lbfgs	yes	solved	1.0555	14.9127
FigureEight	twintrack	ipopt-exact	yes	solved	12.4626	16.5690
FigureEight	twintrack	ipopt-lbfgs	no	iteration limit	521.8558	NaN
FigureEight	bus	ipopt-exact	yes	solved	9.9866	22.8420
FigureEight	bus	ipopt-lbfgs	no	iteration limit	73.1088	NaN

Table 7.16: Hessian approximation benchmark results by track, vehicle model, and transcription variant.

Chapter 8

Conclusion

All goals were achieved. More investigation would be nice into differentiate orthogonal polynomials, Chebyshev type I. polynomials are often used with high order due to their good suppression of Runge phenomena. One thing which was not a goal but, would be super useful if i managed to do a validation on real world data. Validation of sensitivity analysys. I wanted to do it for a long time but there was not enough of it.

Julia was interesting language to learn, it offers blazing fast execution, but that comes with caveats. Debugging is way more cmplicated. When working on a smaller project, the debugging is not such a pain, but when the project is bigger debugging is not user friendly. Debugging in python and MATLAB is way easier. Julias selling point is that it is just in time cmpiled language, rivalling C. That is true BUT you have to know what you are doing, if you dont declare types in functions it is as slow as python.

At the begging of this project I wanted a more complex model, I wanted to go deeper with complexity than my bachelor thesis. That is add pitch, roll and add suspension with damper and spring for each wheel. Also this simulation does not account for gearboxes with discrete stages, and no combustion engine.

Believing 100% this tool and designing car according to it would most likely lead to undriveable vehicle, as there is no force pushing for stability driveability in NLP [60] deals also with this. The optimal driving created by NLP is unachievable by real driver, as the vehicle is on its limit and one mistake woul overload tires and cause failure.

Track model is also heavily simplified, neglecting slope and bank of the track. Accumulator energy constraints I have modeled them but not verified that they work, this is also needed. Motor constraints for softmax

EXAMDELS MAY HAVE BETTER SUPPORT FOR DENSE NONLINEAR EXPRESSIONS< JUMP DOESNT HAVE IT< CASADI HAS IT

valdiation of sensitivity analysis would be good.



Appendices

Appendix A

Bibliography

- [1] 2026 Class Project: Race Car Minimum Lap Time - AE6310 Course Notes. https://sm-dogroup.github.io/ae6310/jupyter/2026_project/race_car_problem.html.
- [2] A Computationally Efficient Framework for Free-trajectory Minimum-lap-time Optimization of Racing Cars. <https://arxiv.org/html/2511.13522v1>.
- [3] OptimumLap | OptimumG.
- [4] Solving ordinary differential equations I. nonstiff problems. *Mathematics and Computers in Simulation*, 29(5):447, October 1987.
- [5] Solving ordinary differential equations I. nonstiff problems. *Mathematics and Computers in Simulation*, 29(5):447, October 1987.
- [6] Does anybody know OpenMDAO - Specific Domains / Optimization (Mathematical). <https://discourse.julialang.org/t/does-anybody-know-openmdao/122840>, November 2024.
- [7] Coin-or/CppAD. COIN-OR Foundation, May 2026.
- [8] acados Developers. acados documentation: real-world examples. https://docs.acados.org/real_world_examples/index.html, 2026. Accessed: 2026-05-14.
- [9] Yunus Agamawi and Anil Rao. CGPOPS: A C++ Software for Solving Multiple-Phase Optimal Control Problems Using Adaptive Gaussian Quadrature Collocation and Sparse Nonlinear Programming. *ACM Transactions on Mathematical Software*, 46, May 2020.
- [10] P.R. Amestoy, I. S. Duff, J. Koster, and J.-Y. L’Excellent. A fully asynchronous multifrontal solver using distributed dynamic scheduling. *SIAM Journal on Matrix Analysis and Applications*, 23(1):15–41, 2001.
- [11] Joel A E Andersson, Joris Gillis, Greg Horn, James B Rawlings, and Moritz Diehl. CasADi – A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1):1–36, 2019.

- [12] Uri M. Ascher, Robert M. M. Mattheij, and Robert D. Russell. *Numerical Solution of Boundary Value Problems for Ordinary Differential Equations*. Classics in Applied Mathematics. SIAM, 1995.
- [13] Jean-Paul Berrut and Lloyd N. Trefethen. Barycentric Lagrange Interpolation. *SIAM Review*, 46(3):501–517, January 2004.
- [14] Mathieu Besançon, Joaquim Dias Garcia, Benoît Legat, and Akshay Sharma. Flexible differentiable optimization via model transformations. *INFORMS Journal on Computing*, 36(2):456–478, 2023.
- [15] John T. Betts. *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming*. Society for Industrial and Applied Mathematics, second edition, January 2010.
- [16] Forbes T. Brown. *Engineering System Dynamics: A Unified Graph-Centered Approach, Second Edition*. CRC Press, Boca Raton, 2 edition, August 2006.
- [17] CasADi Developers. CasADi – an open-source tool for nonlinear optimization and algorithmic differentiation. <https://web.casadi.org/>, 2026. Accessed: 2026-05-14.
- [18] D. Casanova. On minimum time vehicle manoeuvring: The theoretical optimal lap. November 2000.
- [19] Richard Ciglánský. Analysis of vehicle components and their impact on lap-time.
- [20] Richard Ciglánský. SLapSim benchmark animations. YouTube playlist, https://youtube.com/playlist?list=PL_TqVFNheNEL-B8Fof0HaNX4ybwSa8F1H, 2026. Accessed 2026-05-21.
- [21] COIN-OR Foundation. Ipopt options reference. <https://coin-or.github.io/Ipopt/OPTIONS.html>, 2026. Accessed: 2026-05-18.
- [22] Daniel Cortild, J Haastert, A Sanabria, and F Voronine. *A Brief Review of Automatic Differentiation*. March 2023.
- [23] Christopher L. Darby, William W. Hager, and Anil V. Rao. An hp -adaptive pseudospectral method for solving optimal control problems. *Optimal Control Applications and Methods*, 32(4):476–502, July 2011.
- [24] Byron Ehle. On Padé approximations to the exponential function and A-stable methods for the numerical solution of initial value problems.
- [25] T.M. El-Gindy, H.M. El-Hawary, M.S. Salim, and M. El-Kady. A Chebyshev approximation for solving optimal control problems. *Computers & Mathematics with Applications*, 29(6):35–45, 1995.

- [26] Rob Falck, Justin S. Gray, Kaushik Ponnappalli, and Ted Wright. Dymos: A Python package for optimal control of multidisciplinary systems. <https://openmdao.org/dymos/docs/1.9.0/>, 2024. Version 1.9.0, accessed 2026-05-20.
- [27] Robert Falck, Justin S. Gray, Kaushik Ponnappalli, and Ted Wright. dymos: A python package for optimal control of multidisciplinary systems. *Journal of Open Source Software*, 6(59):2809, 2021.
- [28] Liang Fang, Stefan Vandewalle, and Johan Meyers. A parallel-in-time multiple shooting algorithm for large-scale PDE-constrained optimal control problems. *Journal of Computational Physics*, 452:110926, 2022.
- [29] Divya Garg, Michael Patterson, William Hager, Anil Rao, David R Benson, and Geoffrey T Huntington. An overview of three pseudospectral methods for the numerical solution of optimal control problems.
- [30] Divya Garg, Michael Patterson, William W. Hager, Anil V. Rao, David A. Benson, and Geoffrey T. Huntington. A unified framework for the numerical solution of optimal control problems using pseudospectral methods. *Automatica*, 46(11):1843–1851, November 2010.
- [31] Nick Gould, Dominique Orban, and Philippe Toint. CUTEst: A Constrained and Unconstrained Testing Environment with safe threads for Mathematical Optimization. *Computational Optimization and Applications*, 60, July 2014.
- [32] Justin S. Gray, John T. Hwang, Joaquim R. R. A. Martins, Kenneth T. Moore, and Bret A. Naylor. OpenMDAO: An open-source framework for multidisciplinary design, analysis, and optimization. *Structural and Multidisciplinary Optimization*, 59(4):1075–1104, April 2019.
- [33] Sebastien Gros and Moritz Diehl. Numerical Optimal Control (Draft).
- [34] A. Harten, B. Engquist, S. Osher, and S. R. Chakravarthy. Uniformly high order accurate essentially non-oscillatory schemes, III. *Journal of Computational Physics*, 131(1):3–47, 1997.
- [35] Himanshu Hatwal and E. C. Mikulcik. Some inverse solutions to an automobile path-tracking problem with input control of steering and brakes. *Vehicle System Dynamics*, 15:61–71, 1986.
- [36] Pacejka H.B. *Tyre and Vehicle Dynamics*. Butterworth-Heinemann, 2nd ed edition, 2006.
- [37] Alexander Heilmeier, Maximilian Geisslinger, and Johannes Betz. A quasi-steady-state lap time simulation for electrified race cars. In *2019 Fourteenth International Conference on Ecological Vehicles and Renewable Energies (EVER)*. IEEE, May 2019.

- [38] HSL. A collection of Fortran codes for large scale scientific computation. <http://www.hsl.rl.ac.uk/>, 2011.
- [39] IPG Automotive. CarMaker. <https://www.ipg-automotive.com/solutions/product-portfolio/carmaker>, 2026. Accessed: 2026-05-18.
- [40] Shuang Li* Jisong Zhao. Adaptive mesh refinement method for solving optimal control problems using interpolation error analysis and improved data compression.
- [41] Joris Gillis. IPOPT on steroids: Fatrop solver in CasADi, June 2024.
- [42] Julia Discourse. 90% of IPOPT run time is spent in Hessian-constraints evaluation, very little time in linear solver — is it normal? <https://discourse.julialang.org/t/90-of-ipopt-run-time-is-spent-in-hessian-constraints-evaluation-very-little-time/136708>, 2025. Accessed: 2026-04-19.
- [43] JuMP Developers. Performance problem with deeply nested nonlinear expressions. <https://github.com/jump-dev/JuMP.jl/issues/4024>, 2026. GitHub issue #4024, Accessed: 2026-05-13.
- [44] Matthew Kelly. Matthew kelly — personal website. <https://www.matthwepeterkelly.com/index.html>. Accessed: 2026-05-15.
- [45] Matthew Kelly. Trajectory optimization: cannon example. <https://www.matthwepeterkelly.com/tutorials/trajectoryOptimization/cannon.html>. Accessed: 2026-05-15.
- [46] Matthew Kelly. Introduction to trajectory optimization. YouTube, <https://www.youtube.com/watch?v=wlkRYMVUZTs>, 2017. Accessed: 2026-05-15.
- [47] Matthew Kelly. An introduction to trajectory optimization: How to do your own direct collocation. *SIAM Review*, 59(4):849–904, 2017.
- [48] Matthew Kelly. An Introduction to Trajectory Optimization: How to Do Your Own Direct Collocation. *SIAM Review*, 59(4):849–904, January 2017.
- [49] Matthew Kelly. Trajectory optimization terminology. <https://www.matthwepeterkelly.com/tutorials/trajectoryOptimization/terminology.html>, 2026. Accessed: 2026-05-14.
- [50] Fengjin Liu, William W. Hager, and Anil V. Rao. Adaptive Mesh Refinement Method for Optimal Control Using Decay Rates of Legendre Polynomial Coefficients. *IEEE Transactions on Control Systems Technology*, 26(4):1475–1483, July 2018.

- [65] Sungho Shin, Carleton Coffrin, Kaarthik Sundar, and Victor M Zavala. Graph-based modeling and decomposition of energy infrastructures. *IFAC-PapersOnLine*, 54(3):693–698, 2021.
- [66] Sungho Shin, François Pacaud, and Mihai Anitescu. Accelerating optimal power flow with GPUs: SIMD abstraction of nonlinear programs and condensed-space interior-point methods, 2023.
- [67] Sungho Shin, François Pacaud, and Mihai Anitescu. How to solve on the GPU a JuMP model — MadNLP.jl documentation. <https://madsuite.org/MadNLP.jl/stable/tutorials/gpu/>, 2026. Accessed 2026-05-21.
- [68] Lloyd N Trefethen. Approximation Theory and Approximation Practice.
- [69] Lloyd N. Trefethen. *Spectral Methods in Matlab*. Software, Environments, Tools. SIAM: Society for Industrial and Applied Mathematics, 2001.
- [70] Charlie Vanaret and Sven Leyffer. *Implementing a unified solver for nonlinearly constrained optimization*. November 2025.
- [71] Lander Vanroye, Ajay Sathya, Joris De Schutter, and Wilm Decré. FATROP : A Fast Constrained Optimal Control Problem Solver for Robot Trajectory Optimization and Control, July 2023.
- [72] Robin Verschueren, Gianluca Frison, Dimitris Kouzoupis, Jonathan Frey, Niels van Duijkeren, Andrea Zanelli, Branimir Novoselnik, Thivaharan Albin, Rien Quirynen, and Moritz Diehl. acados – a modular open-source framework for fast embedded optimal control. *Mathematical Programming Computation*, 2021.
- [73] Matthew J Weinstein and Anil V Rao. Algorithm 984: Adigator, a toolbox for the algorithmic differentiation of mathematical functions in matlab using source transformation via operator overloading. *ACM Transactions on Mathematical Software (TOMS)*, 44(2):21, 2017.
- [74] Matthew E. Wilhelm and Matthew D. Stuber. EAGO.jl: easy advanced global optimization in Julia. *Optimization Methods and Software*, 37(2):425–450, 2022.
- [75] Hendrik Wilka and Jens Lang. Adaptive hp-Polynomial Based Sparse Grid Collocation Algorithms for Piecewise Smooth Functions with Kinks, May 2025.
- [76] Andreas Wächter and Lorenz T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, March 2006.